

UNIT- III

3

Java Servlets and XML

Syllabus

Servlet : Servlet architecture overview, A “Hello World” servlet, Servlets generating dynamic content, Servlet life cycle, parameter data, sessions, cookies, URL rewriting, other Servlet capabilities, data storage, Servlets concurrency, databases (MySQL) and Java Servlets.

XML : XML documents and vocabularies, XML declaration, XML Namespaces, DOM based XML processing, transforming XML documents, DTD : Schema, elements, attributes.

AJAX : Introduction, Working of AJAX.

Contents

- 3.1 Introduction to Server Side Programming
- 3.2 Servlet Architecture Overview
- 3.3 A "Hello World" Servlet.....**May-19**, Marks 5
- 3.4 Servlets Generating Dynamic Content
- 3.5 Servlet Life Cycle**March-20**, Marks 5
- 3.6 Servlet API
- 3.7 Parameter Data
- 3.8 Sessions
- 3.9 Cookies **Dec.-19**, Marks 5
- 3.10 URL Rewriting
- 3.11 Other Servlet Capabilities
- 3.12 Data Storage
- 3.13 Servlets and Concurrency
- 3.14 Databases (MySQL) and Java Servlets
- 3.15 XML Documents and Vocabularies.....**Dec.-18, 19, May-19**, Marks 5
- 3.16 XML Declaration

3.17	Rules	
3.18	XML Namespaces	
3.19	Document Type Definition (DTD)	May-18, Marks 5
3.20	Schema	May-18, Marks 5
3.21	DOM based XML Processing	
3.22	Transforming XML Documents	May-19, March-20, Marks 5
3.23	Introduction	
3.24	Working of AJAX	
3.25	XMLHttpRequest Object	
3.26	Coding AJAX Script	Dec.-18, 19, May-18,19 Marks 8
3.27	Multiple Choice Questions	

Part I : Servlet

3.1 Introduction to Server Side Programming

- Server side technology deals with the data submitted by the client.
- Server side program basically **handles the request** made by the client. This program then **prepares the response** for the client.
- The server side programming also act as a link between backend database and front end client side programming.
- Various server side technologies include – servlet, JSP, ASP.NET, PHP and so on.

Difference between client side and server side scripting

Sr. No.	Server side scripting	Client side scripting
1.	The server side scripting is used to create the web pages that provide some services.	The client side scripting is used to create the web pages as a request or response to server. These pages are displayed to the user on web browser.
2.	These scripts generally run on web servers.	These scripts generally run on web browser.
3.	A user's request is fulfilled by running a script directly on the web server to generate dynamic HTML pages. This HTML is then sent to the client browser.	The processing of these scripts takes place on the end users computer. The source code is transferred from the web server to the users computer over the internet and run directly in the browser.
4.	Uses : Processing of user request, accessing to databases.	Uses : Making interactive web pages, for interacting with temporary storages such as cookies or local storage, sending request to server and getting the response and displaying that response in web browser.
5.	Example : PHP, ASP.NET, nearly all the programming languages including C++, Java and C#	Example : HTML, CSS, JavaScript(primarily)

3.2 Servlet Architecture Overview

- Servlets are basically the Java programs that run on server. These are the programs that are requested by the XHTML documents and are displayed in the browser window as a response to the request.

- The servlet class is instantiated when web server begins the execution.
- The execution of servlet is managed by **servlet container**, such as Tomcat.
- The servlet container is used in java for dynamically generate the web pages on the server side. Therefore the servlet container is the part of a web server that interacts with the servlet for handling the dynamic web pages from the client.
- Servlets are most commonly used with HTTP (i.e. Hyper Text Transfer Protocol) hence sometimes servlets are also called as "HTTP Servlet".
- The main purpose of servlets is to add up the functionality to a web server.

Working of How Servlet Works ?

- Before learning the actual servlet programming it is very important to understand how servlet works.

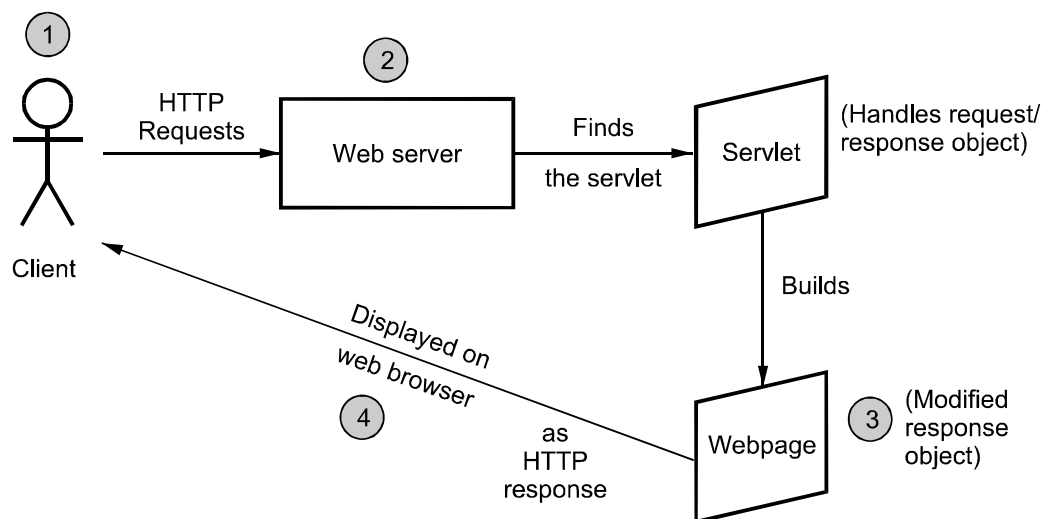


Fig. 3.2.1 How servlet works ?

1. When a client make a request for some servlet, he/she actually uses the **Web browser** in which request is written as a URL.
2. The web browser then sends this request to **Web server**. The web server first finds the requested servlet.
3. The obtained servlet gathers the relevant information in order to satisfy the client's request and builds a **web page** accordingly.
4. This web page is then **displayed** to the client. Thus the request made by the client gets satisfied by the servlets.

3.2.1 Need and Advantages

- Servlets can process and store the data submitted by an HTML form.
- Servlets are useful for providing the dynamic contents. For example retrieving and updating the databases.
- Servlets can be used in the cookies. Cookies are small programs which can make use of the information submitted on currently accessed web pages.
- Similarly servlets are used in session tracking. The session tracking are the useful programs that keeps track of all previously accessed the web pages.

Advantages

- The servlets are very **efficient** in their performance and get executed in the address space of the belonging web server.
- The servlets are **platform independent** and can be executed on different web servers.
- The servlets working is based on **Request-Response**. Any HTML form can take the user input and can forward this input to the servlet. The servlets are then responsible to communicate with the back-end database and manipulate the required business logic. These servlets embedded on the web servers using **Servlets API**.
- Servlets provide a way to generate the **dynamic document**. For instance : A servlet can display the information of current user logged in, his logging time, his last access, total number of access he made so far and so on.
- **Multiple users** can keep a co-ordination for some application among themselves using servlets.
- Using servlets **multiple requests** can be synchronized and then can be concurrently handled.

Disadvantages

- Servlet is a mixture of java skills and web related HTML skills. Hence it is difficult to write servlets.
- It slows down the application execution.
- The java runtime environment is required on the server, if you want to execute servlet application.

3.3 A "Hello World" Servlet

SPPU : May-19, Marks 5

- When we write a servlet program, it is necessary to - i) Either implement **Servlet** interface or ii) Extend a class that implements **Servlet** interface.

- While implementing **Servlet** interface we must include **javax.servlet** package. Hence the first line in our servlet program must be,

```
import javax.servlet.*;
```

- **GenericServlet** class is a predefined implementation of **Servlet** interface. Hence we can extend **GenericServlet** class in our servlet program. Similarly, the **HttpServlet** class is a child class of **GenericServlet** class, hence we can extend this class as well while writing the servlet program.
- Hence following are the two ways by which we can write servlet program

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
public class Test extends
GenericServlet
{
    //body of servlet
}
```

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
public class Test extends
HttpServlet
{
    //body of servlet
}
```

- The servlet gets the request from the client for some service. The servlet then processes the request and sends the response back to the client. In order to handle these issues **HttpServletRequest** and **HttpServletResponse** are used in servlet program.
- These requests are handled with the help of some methods that are popularly known as **methods of HttpServlet**. These methods are as follows,

Method	Purpose
doGet	This method handles the HTTP GET request
doPost	This method handles the HTTP POST request
doPut	This method handles the HTTP Put request.
doDelete	This method handles the DELETE request.

The doGet and doPost methods

- The **doGet** method requests the data from the source.
- The **doPost** method submits the processed data to the source.
- The protocol of **doGet** method is as follows,

```
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException,IOException
```

- The **ServletException** and **IOException** are thrown to handle the Servlet problems gracefully.

- The **HttpServletRequest** request : Contain the client request made by client.
- The **HttpServletResponse** response : Contains the response made by servlet back to the client.
- The protocol of doPost method is same as doGet method. It is as follows -

```
protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException,IOException
```
- The GET request is more efficient than the POST request.
- The GET request is less secure than the POST request.

How to Write Servlet Program ?

Open notepad and write the first servlet code to display greeting messages. It is as follows,

FirstServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class FirstServlet extends HttpServlet
{
    public void doGet(HttpServletRequest request,HttpServletResponse response)
    throws IOException,ServletException
    {
        response.setContentType("text/html");
        PrintWriter out=response.getWriter();
        out.println("<html>");
        out.println("<head>");
        out.println("<title>My First Servlet</title>");
        out.println("<body>");
        out.println("<h1>Hello How are U?</h1>");
        out.println("<h2>I am enjoying this Servlet Application</h2>");
        out.println("<h3>See You later!</h3>");
        out.println("</body>");
        out.println("</html>");
    }
}
```

Program Explanation :

- In the above program, we have imported following files,

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
```

- Out of these files **java.io** package is useful for taking care of I/O operations.
- The **javax.servlet** and **javax.servlet.http** are important packages containing the **classes** and **interfaces** that are required for the operation of servlets. The most commonly used interface from **javax.servlet** package is **Servlet**. Similarly most commonly used class in this package is **GenericServlet**. The **ServletRequest** and **ServletResponse** are another two commonly used interfaces defined in **javax.servlet** package.
- In the **javax.servlet.http** package **HttpServletRequest** and **HttpServletResponse** are two commonly used interfaces. The **HttpServletRequest** enables the servlet to read data from the HTTP request and **HttpServletResponse** enables the servlet to write data to HTTP response. The **cookie** and **HttpServlet** are two commonly used classes that are defined in this package.
- We have given class name **FirstServlet** which should be derived from the class **HttpServlet**. (Sometimes we can derive our class from **GenericServlet**).
- Then we have defined **doGet** method to which the HTTP request and response are passed as parameters. The commonly used basic exceptions for the servlets are **IOException** and **ServletException**.
- The MIME type is specified as using the **setContentType()** method. This method sets the content type for the HTTP response to type. In this method "text/html" is specified as the MIME type. This means that the browser should interpret the contents as the HTML source code.
- Then an output stream is created using **PrintWriter()**. The **getWriter()** method is used for obtaining the output stream. Anything written to this stream is sent to the client as a response. Hence using the object of output stream 'out', we can write the HTML source code in **println** method as HTTP response.

How to execute Servlet program ?

Step 1 : Compile the above program using the javac command at command prompt.

```
D:\test>javac FirstServlet.java
```

The class file for this program gets generated.

Step 2 : Before running any servlet program, it is necessary to have

1. JDK installed
2. Tomcat installed.
3. Class path for above two packages must be set.

For Tomcat installation, I prefer to install the package XAMPP. The XAMP contains a directory for tomcat. The XAMPP package contains Apache Web server, MySQL, PHP and Perl support. It can work on both Windows and Linux operating System.

Step 3 : Copy the class file generated in Step 1 to the path

C : \xampp\tomcat\webapps\examples\WEB-INF\classes

Step 4 : Go to the directory

C : \xampp\tomcat\webapps\examples\WEB-INF

Open **web.xml** file and edit it as follows

```
<servlet>
  <servlet-name>FirstServlet</servlet-name>
  <servlet-class> FirstServlet </servlet-class>
</servlet>
...
...
...
<servlet-mapping>
  <servlet-name> FirstServlet </servlet-name>
  <url-pattern>/servlet/ FirstServlet </url-pattern>
</servlet-mapping>
```

The **web.xml** file is popularly known as **deployment descriptor**. This is basically the configuration file that describes how to map the URL of web application to servlet.

The diagram shows the web.xml deployment descriptor with annotations for internal and external alias names. The annotations are as follows:

- Internal alias Name:** Points to the `<servlet-name>FirstServlet</servlet-name>` tag in the `<servlet>` block.
- Actual Name of server:** Points to the `<servlet-class> FirstServlet </servlet-class>` tag in the `<servlet>` block.
- Internal alias Name:** Points to the `<servlet-name> FirstServlet </servlet-name>` tag in the `<servlet-mapping>` block.
- External alias Name:** Points to the `<url-pattern>/servlet/ FirstServlet </url-pattern>` tag in the `<servlet-mapping>` block.

```
<servlet>
  <servlet-name>FirstServlet</servlet-name>
  <servlet-class> FirstServlet </servlet-class>
</servlet>
...
...
...
<servlet-mapping>
  <servlet-name> FirstServlet </servlet-name>
  <url-pattern>/servlet/ FirstServlet </url-pattern>
</servlet-mapping>
```

The Servlet comes with two alias names, internal and external. The internal name is used by the Tomcat and the external name is given (to be written in `<FORM>` tag of HTML file) to the client to invoke the Servlet on the server. That is, there exists alias to

alias. All this is for security. Observe, the names are given in two different XML tags, in the web.xml file, to make it difficult for hacking.

To invoke the **FirstServlet** Servlet, the client calls the server with the name **servlet/FirstServlet**.

When **servlet/FirstServlet** call reaches the server, the Tomcat server opens the **web.xml** file to check the deployment particulars. Searches such a `<servlet-mapping>` tag that matches **servlet/FirstServlet**. **servlet/FirstServlet** is exchanged with **FirstServlet**.

Then, searches such a `<servlet>` tag that matches **FirstServlet** and exchanges with **FirstServlet class**. Now the server, loads **FirstServlet** Servlet, executes and sends the output of execution as response to client.

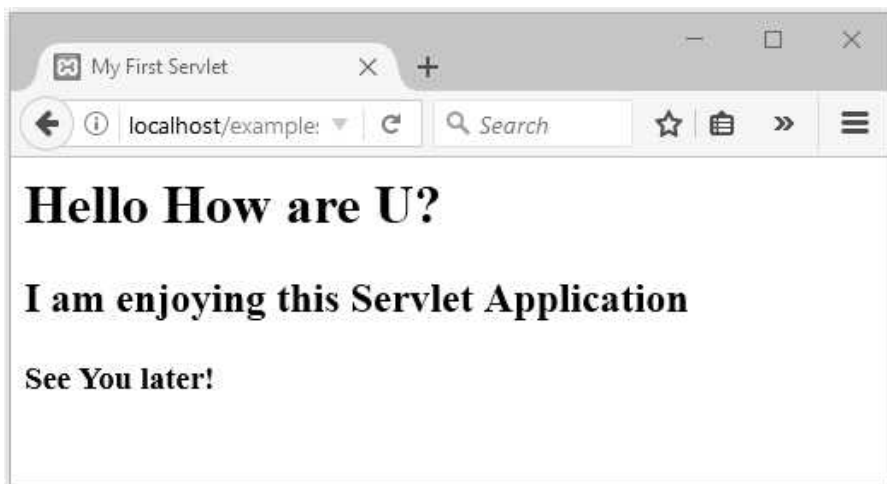
Step 5 : Start tomcat and xampp

Step 6 : Open web browser and type the command

`http://localhost/examples/servlet/FirstServlet`

The output will be

Output



Review Question

1. Write a Java servlet program which will display "welcome to servlet" message.

SPPU : May-19, Marks 5

3.4 Servlets Generating Dynamic Content

- In modern web technology, the web site must be able to generate dynamic contents as per the requirements. Servlets can generate static, dynamic or interactive web pages.

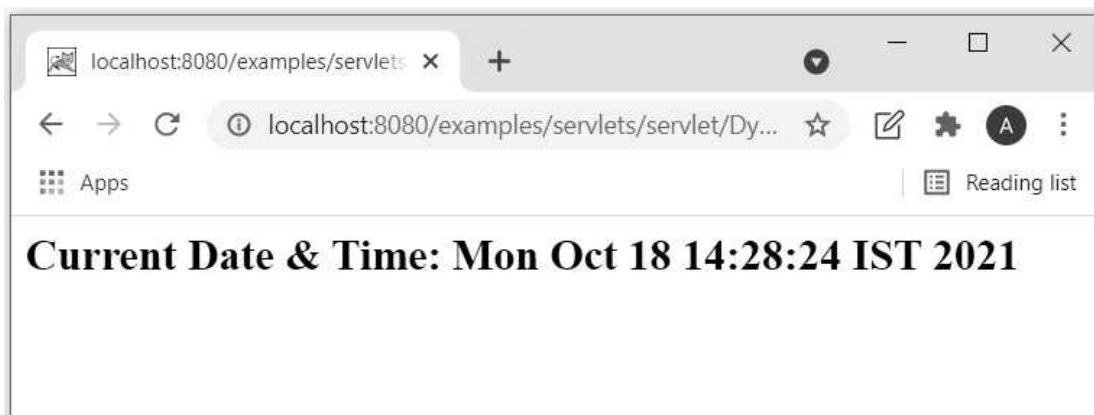
- Following is a simple servlet program that displays the current date and time. Note that the time is continuously changing and on refreshing the same page, we can note down the changes.

Example Program

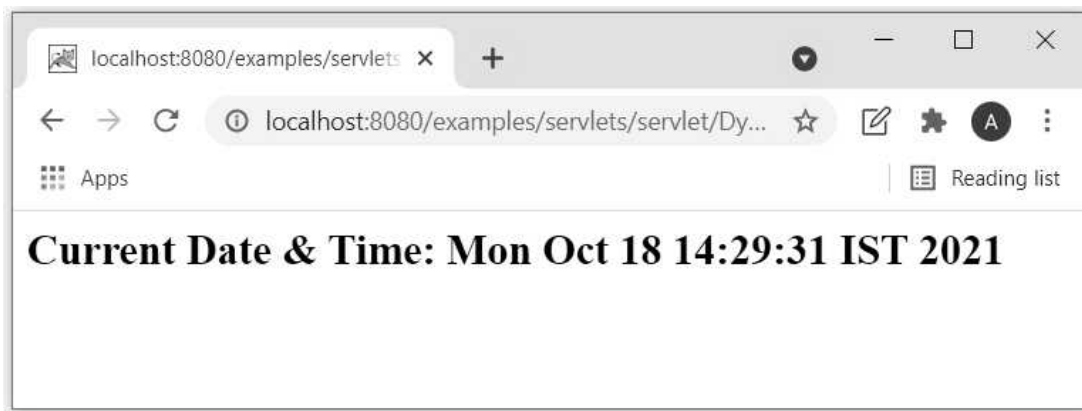
```
import java.io.*;
import javax.servlet.*;

public class DynamicServletDemo extends GenericServlet
{
    //implement service()
    public void service(ServletRequest req, ServletResponse res)
        throws IOException, ServletException
    {
        //set response content type
        res.setContentType("text/html");
        //get stream obj
        PrintWriter out = res.getWriter();
        java.util.Date date = new java.util.Date();
        out.println("<h2>"+"Current Date & Time: " + date.toString()+"</h2>");
        //close stream object
        out.close();
    }
}
```

Output



Press F5 and refresh the current page, you will get the some changes in the output.



Program Explanation : In above program,

1. We have used Date() method of java.util class that is intended to display current date and time.
2. The date is displayed using the **date.toString()** method.

3.5 Servlet Life Cycle

SPPU : March-20, Marks 5

In the life cycle of servlet there are three important methods. These methods are,

1. **Init**
2. **Service**
3. **Destroy**

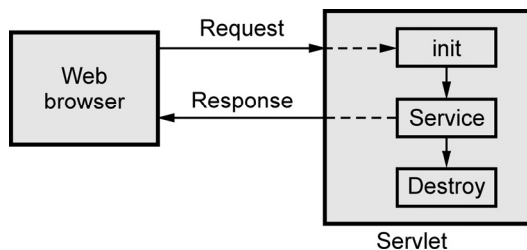


Fig. 3.5.1 Life cycle of servlet

- The client enters the URL in the web browser and makes a request. The browser then generates the HTTP request and sends it to the Web server. (Refer Fig. 3.5.1)
- Web server maps this request to the corresponding servlet.
 1. **Init () Method** : The server basically invokes the **init()** method of servlet. This method is called only when the servlet is loaded in the memory for the first time.

Using this method initialization parameters can also be passed to the servlet in order to configure itself.

2. **service () Method** : Server can invoke the service for particular HTTP request using **service()** method. The servlets can then read the data provided by the HTTP request with the help of **service()** method.
3. **destroy () Method** : Finally server unloads the servlet from the memory using the **destroy()** method.

Example Program

Following is a simple servlet in which these three methods viz. **init()**, **service()** and **destroy()** are used -

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;

public class LifeCycle extends GenericServlet
{
    public void init(ServletConfig config) throws ServletException
    {
        System.out.println("init");
    }

    public void service(ServletRequest request, ServletResponse response)
    throws ServletException, IOException
    {
        System.out.println("from service");
        PrintWriter out = response.getWriter();
        out.println("Twinkle Twinkle Little Stars.");
        out.print("How I wonder what you are.");
    }

    public void destroy()
    {
        System.out.println("destroy");
    }
}
```

Program Explanation : In the above program,

1. We have first of all used an **init()** method to which object of the **ServletConfig** interface is passed. This interface allows the servlet to get the initialization parameters.
2. Then we have used the **service()** method to which the **ServletRequest** and

ServletResponse parameters are passed for making the HTTP request and getting the HTTP response from the servlet.

3. The output stream is created using **getWriter()** and along this output stream some messages are written. These messages will then be displayed on the web browser when the servlet gets executed.
4. Finally the **destroy()** function is written in order to unload the servlet from the memory.

Review Question

1. Explain the lifecycle of servlet. Write a simple servlet to print "Hello, Good Day".

SPPU : March-20, Marks 5

3.6 Servlet API

There are two packages used to implement the **Servlet**. These packages are,

1) javax.servlet

2) javax.servlet.http

The classes and interfaces defined in these packages help in implementing the servlet.

Let us discuss them in detail -

3.6.1 The javax.servlet Package

- This package is used to implement the servlet. Out of many other interfaces and classes defined in this package the most important interface is **Servlet**. All the servlet must implement this interface. The interfaces and classes that are defined in this package are enlisted below -

3.6.1.1 Interfaces

Interface	Description
Servlet	This interface defines all the life cycle methods.
ServletConfig	This interface obtains the initialization parameters.
ServletContext	Using this interface the events can be logged.
ServletRequest	This interface is useful in reading the data from the client request.
ServletResponse	This interface is useful in writing the data to the client response.

1. The Servlet Interface

- This interface defines all the life cycle methods such as **init()**, **destroy()** and **service()**. All the servlets must implement this interface. Some of the methods provided by this interface are -

Method	Description
void init(ServletConfig s)	This method is called for initialising the servlet. The initialisation parameter is obtained from the ServletConfig interface.
void destroy()	This method is called when the servlet has to be unloaded.
ServletConfig getServletConfig()	This method is used to obtain the initialisation parameters.
void Service(ServletRequest req, ServletResponse res)	This method is used to implement the service that a servlet should provide. The clients request is processed and a response is given.
String getServletInfo()	This method is used to obtain description of the servlet.

2. The ServletConfig Interface

- This interface is used to obtain the initialisation parameters. Various methods that are used by this interface are -

Method	Description
String getServletName()	This method is used to obtain the name of the servlet.
String getInitParameter(String p)	This method returns the value of the parameter <i>p</i>
Enumeration getInitParameterNames()	This method returns the names of initialisation parameters.
ServletContext getServletContext()	This method returns the context for the servlet.

3. The ServletContext Interface

- This is the interface that enables the servlet to log events and access information about their environment. Various methods of this interface are -

Method	Description
Object getAttribute(String attribute_name)	The value of the attribute <i>attribute_name</i> in the current session is returned.

<code>void setAttribute(String <i>attribute_name</i>, object <i>value</i>)</code>	The <i>attribute_name</i> is passed to the object <i>value</i> .
<code>String getServerInfo()</code>	This method returns the information about the server.
<code>String log(String <i>str</i>)</code>	Writes the <i>str</i> to the servlet log.
<code>String getMimeType(String <i>file</i>)</code>	It returns the MIME type of the file.

4. The ServletRequest Interface

- The **ServletRequest** interface helps in gathering the information about the client. Various methods in ServletRequest are -

Method	Description
<code>Object getAttribute(String <i>attribute_name</i>)</code>	The value of the attribute <i>attribute_name</i> in the current session is returned.
<code>int getContentLength()</code>	It returns the content size of the request.
<code>String getContentType()</code>	It returns the type of the request.
<code>getInputStream()</code>	This method is used to read the binary data from the request.
<code>getProtocol()</code>	Returns the name of the protocol used.
<code>getReader()</code>	This method is useful for reading the text from the request.
<code>getServerName()</code>	It returns the name of the server on which the servlet is running.
<code>int getServerPort()</code>	It returns the port number of the server.

5. The ServletResponse Interface

- This interface helps the servlet to formulate the response for the client's request. Various methods of this interface are -

Method	Description
<code>String getCharacterEncoding()</code>	This method returns the character encoding.
<code>ServletOutputStream getOutputStream()</code>	This method returns outputstream which is used to write the data for responding.

<code>PrintWriter getWriter()</code>	This method is used to write the character data to the response.
<code>void setContentLength(int size)</code>	This method sets the length of the content equal to the <i>size</i> .
<code>void setContentType(String Type)</code>	This methods sets the type of the content

3.6.1.2 Classes

- The **javax.servlet** package contains following classes.

Class	Description
<code>GenericServlet</code>	This class implements the Servlet and ServletConfig interfaces.
<code>ServletInputStream</code>	This class provides the input stream for reading the client's request.
<code>ServletOutputStream</code>	This class provides the output stream for writing the client's response.
<code>ServletException</code>	This class is used to raise the exception when an error occurs.

1. The **GenericServlet** class

- This class is useful for implementing the life cycle methods. It implements the **Servlet** and **ServletConfig** interfaces. It is called "Generic" because it does not assume that the protocol it will process will be HTTP. If we want to write our own protocol, then we must extend this class.

2. The **ServletInputStream** class

- The **ServletInputStream** class extends the **InputStream**. This class provides an input stream which is used by a servlet for reading the data from a client request. For reading the bytes from a client the method used is **readLine**.

3. The **ServletOutputStream** class

- The **ServletOutputStream** class extends the **OutputStream**. This class provides an output stream which is used by a servlet for writing the data for a client response. It also defines **print** and **println** method.

4. The **ServletException** class

- The **javax.servlet** defines two servlets **ServletException** and **UnavailableException**

3.6.2 The javax.servlet.http Package

- The javax.servlet.http package contains a number of interfaces and classes and their functionality makes it easy to build servlets. These servlets then work with HTTP requests and responses.

3.6.2.1 Interface

- The following table describes the interfaces that are provided in this package

Interface	Description
HttpSession	The session data can be read or written using this interface.
HttpServletRequest	The servlet can read the information from the HTTP request using this interface.
HttpServletResponse	The servlet can write the data to HTTP response using this interface.
HttpSessionBindingListener	This interface tells the object about its binding with the particular session.

1. The HttpSession

- At every HTTP session, the state information is gathered. The servlet can read or write this information using **HttpSession** interface. This interface is implemented by the server.
- Various methods used by this interface are as given below -

Method	Description
String getId()	This method returns the session ID.
Object getAttribute(String <i>attribute_name</i>)	The value of the attribute <i>attribute_name</i> in the current session is returned.
Enumeration getAttributeNames()	Returns the attribute names.

2. The HttpServletRequest

- The **HttpServletRequest** is used to obtain the information from client's HTTP request.

Method	Description
String getMethod()	It returns the HTTP method for the client request.
String getPathInfo()	Returns the path information about the servlet path.
HttpSession getSession()	Returns the current session.

String getHeader(String <i>fields</i>)	It returns the value of header fields.
Cookie [] getCookies()	It returns the information in the cookies in the request made.
String getAuth Type()	It returns the type of Authentication.

3. The HttpServletResponse

- This **HttpServletResponse** interface is used to formulate an HTTP response to the client. Various methods are -

Method	Description
void addCookie(Cookie <i>cookie</i>)	This method is used to add cookie in the response.
String encodeURL(String <i>url</i>)	This method is used to encode the specified URL.
Boolean containsHeader(String <i>f</i>)	This method returns true if the response header contains the field <i>f</i> .
void sendError(int <i>code</i>)	This method sends error code to the client.

4. The HttpSessionBindingListener

- When particular listener gets bound or unbound from a value within **HttpSession** object then this class is required. It extends the **HttpSessionEvent**. Various methods that can be defined by this interface are -

Method	Description
Object getValue()	This function returns the value of bounded or unbounded attribute.
String getName()	This function returns the name being bound or unbound.
HttpSession getSession()	This function returns the session to which the listener can be bound or unbound.

3.6.2.2 Classes

- The following table describes the classes that are provided in this package

Class	Description
Cookie	This class is used to write the cookies.
HttpServlet	It is used when developing servlets that receive and process HTTP requests.
HttpSessionEvent	This class is used to handle the session events.
HttpSessionBindingEvent	When a listener is bound to a value.

1) The Cookie class

- A cookie is a small piece of information that is stored in the client's machine. Sometimes cookies are useful to keep track of the user using the client machine. Various methods used by this class are -

Class	Description
String getValue()	This function returns a value of the cookie.
void setValue(String s)	This function sets the value to the cookie.
String getName()	This function returns the name.

2) The HttpServlet class

- The **HttpServlet** class extends **GenericServlet**. It is used when developing servlets that receive and process HTTP requests. Various methods of this class are -

Class	Description
Void doGet(HttpServletRequest req, HttpServletResponse res)	This method performs HTTP get request.
Void doPost(HttpServletRequest req, HttpServletResponse res)	This method performs HTTP Post request.
Void doPut(HttpServletRequest req, HttpServletResponse res)	This method performs HTTP Put request.
Void service(HttpServletRequest req, HttpServletResponse res)	This method is invoked for processing HTTP request and response.

3) The HttpSessionEvent

- This method encapsulates the session event.

4) HttpSessionBindingEvent

- The **HttpSessionBindingEvent** class extends **HttpSessionEvent**. It is generated when a listener is bound to a value. The **getSession()** method obtains the session to which the listener is being bound or unbound.

3.7 Parameter Data

- Many times we need to pass some information from web browser to Web server. In such case, the HTML document containing the FORM is written which sends the data to the servlet present on the web server.

- To make the form works with Java servlet, we need to specify the following attributes for the <form> tag :
 - i) **method="method name"**: To send the form data as an HTTP POST request to the server.
 - ii) **action="URL/address of the servlet"**: Specifies relative URL of the servlet which is responsible for handling data posted from this form.
- The browser uses two methods to pass this information to web server. These methods are GET Method and POST Method.
 - i) **GET method** : The GET method sends user information along with ? symbol called query string. For instance
`http://localhost/hello?user = aaaa&age = 20`
 - ii) **POST method** : This is the most reliable method of sending user information to the server from HTML form. Servlet handles this request using **doPost** method.

Difference between GET and POST

GET	POST
Using GET request limited amount of information can be sent.	Using POST large amount of information can be sent.
GET request is not secured as information is visible in URL	This is a secured request.
This request is can be bookmarked.	This request can not be bookmarked.
This request is more efficient.	This request is less efficient.

How does servlet read form data ?

Servlet makes use of following three methods to read the data entered by the user on the HTML form

1. **getParameter()** – You call request.getParameter() method to get the value of a form parameter.
2. **getParameterValues()** – Call this method if the parameter appears more than once and returns multiple values, for example checkbox.
3. **getParameterNames()** – Call this method if you want a complete list of all parameters in the current request.

Programming Examples

Example 3.7.1 Write a HTML that shows the following list :

C,C++,JAVA,C#

Define a form that contains a select statement and submit button. If the user selects the java and press the submit the web page displays "The selected language is Java". Write a servlet program using HttpServlet and doGet method.

Solution :

- We will write two source files first one is the HTML file named **test.html** in which the list of all the desired languages is displayed along with the submit button. User has to select the language of his choice and then press the submit button.
- This data made by this request will be received by the servlet named **my_choiceservlet.java** which reads the selected parameter and displays the message about the selection made. The source files are as follows -

test.html

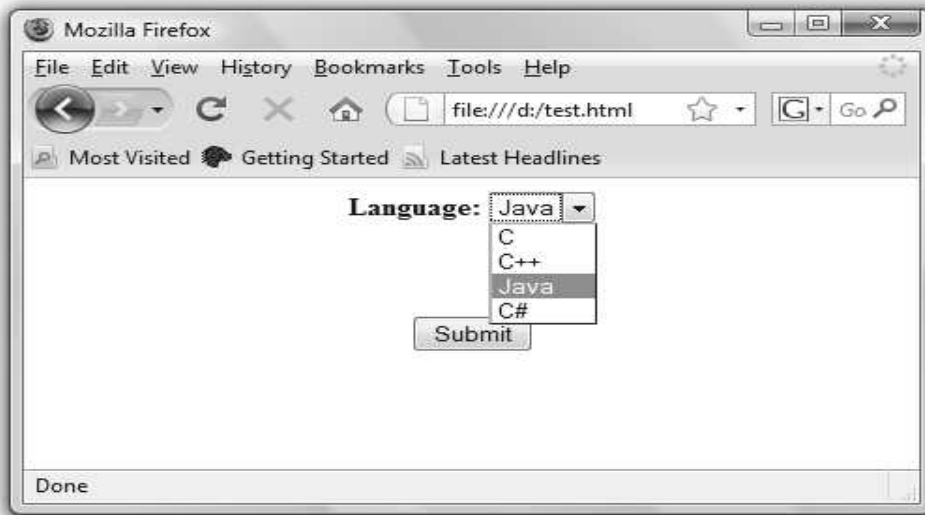
```
<html>
  <body>
    <center>
      <form name="form1" method=GET
        action="http://localhost:4040/servlets-examples/servlet/my_choiceservlet">
        <b>Language:</b>
        <select name="Language" size="1">
          <option value="C">C</option>
          <option value="C++">C++</option>
          <option value="Java">Java</option>
          <option value="C#">C#</option>
        </select>
        <br><br>
        <input type="submit" value="Submit">
      </form>
    </body>
  </html>
```

my_choiceservlet.java

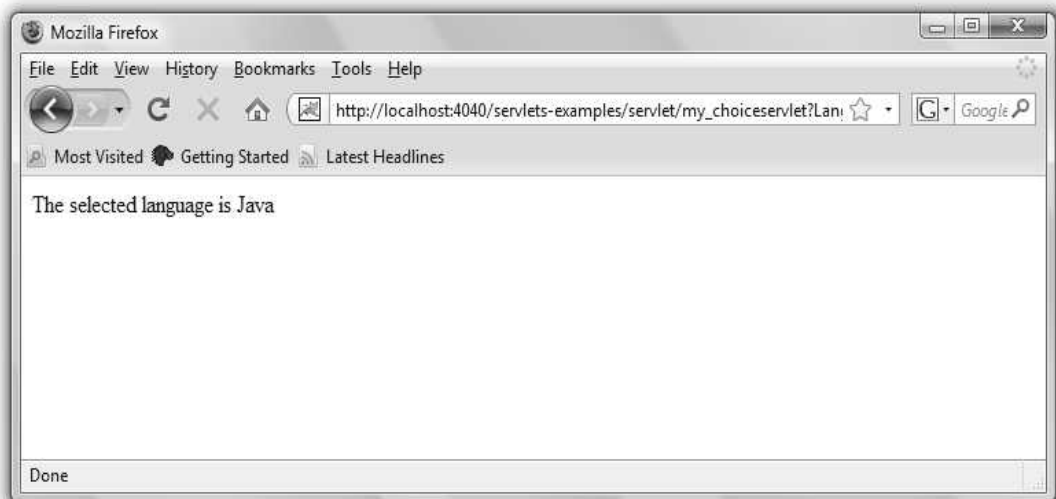
```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class my_choiceservlet extends HttpServlet
{
    public void doGet(HttpServletRequest req,HttpServletResponse res)
```

```
throws ServletException, IOException
{
    String lang=req.getParameter("Language");
    res.setContentType("text/html");
    PrintWriter out=res.getWriter();
    out.println("The selected language is "+lang);
    out.close();
}
}
```



On clicking the submit button we will get following output.



Example 3.7.2 Write HTML form to read user name and password. This data is sent to the servlet. If the correct user name and password is given then welcome him/her by his/her name otherwise display the message for invalid user.

Solution :

Step 1 : Create HTML form for accepting user name and password

Input.html

```
<html>
<head>
</head>
<body>
<form action="http://localhost/examples/servlets/servlet/Welcome" method="get">
User Name:<input type="text" name="uname"/>
<br/>
Password:<input type="password" name="pwd"/>
<input type="submit" value="Submit"/>
</form>
</body>
</html>
```

Step 2 : Create the servlet program to read user name and password and validate it.

Welcome.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class Welcome extends HttpServlet
{
    public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException
    {
        PrintWriter out=res.getWriter();
        res.setContentType("text/html");

        String username=req.getParameter("uname");
        String password=req.getParameter("pwd");
        if ((username=="Ankita")&&(password=="1234"))
            out.print("Welcome "+username);
        else
            out.println("Invalid username");
    }
}
```


Example 3.7.3 Write a servlet which accept two numbers using POST methods and display the maximum of them.

Solution :

Step 1 : The HTML document for inputting two numbers is as follows -

NumbersInput.html

```
<html>
<head>
<body>
<div align="center">
  <br> <br>
  <form action="http ://localhost/examples/servlets/servlet/MaxNumber" method="post">
    Enter First Number :
    <input type="text" value="" name="Number1" size='5'>
    <br/> <br/> Enter Second Number :
    <input type="text" value="" name="Number2" size='5'>
    <br/> <br/> <br/>
    <input type="submit" value="Submit">
  </form>
</div>
</body>
</html>
```

Step 2 : The servlet code that handles the post method and finds the maximum of the two input numbers is as follows -

MaxNumber.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

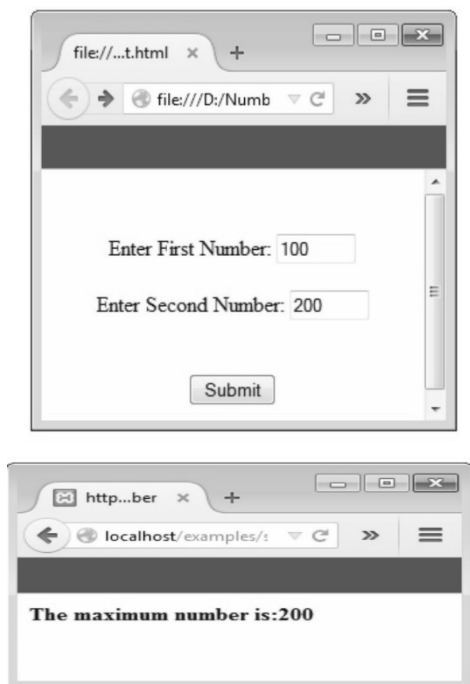
public class MaxNumber extends HttpServlet
{
  protected void doPost(HttpServletRequest req, HttpServletResponse res) throws
ServletException, IOException
  {
    res.setContentType("text/html");
    PrintWriter out=res.getWriter();

    // get request parameters for userID and password
    int a = Integer.parseInt(req.getParameter("Number1"));
    int b = Integer.parseInt(req.getParameter("Number2"));
```

```
    if (a>b)
        out.println("<h4>The maximum number is:" + a + "</h4>");
    else
        out.println("<h4>The maximum number is :"+ b + "</h4>");

}
```

Step 3 : The output is as follows -



Example 3.7.4 Write the code for converting currencies to US Dollar.

Solution :

Step 1 : We will write the simple HTML document, which will create the interface. **The user can select the type of currencies and will enter the value.**

```
<html>
<body>
    <formname="form1"method="post"
action="http://localhost/examples/servlets/servlet/DollarDemo">
    Select the type of Currency:
    <Select name="currency">
        <option value="Euro">Euro</option>
        <option value="Pound">Pound</option>
        <option value="Rupees">Rupees</option>
```

```
</select>
<br/> <br/>
Enter the value:
<input type="text" name="currencyVal" value=""/>
<br/> <br/>
<input type="submit" value="Submit"/>
</form>
</body>
</html>
```

Step 2 : The java servlet for converting the currency value to dollar is as given below -

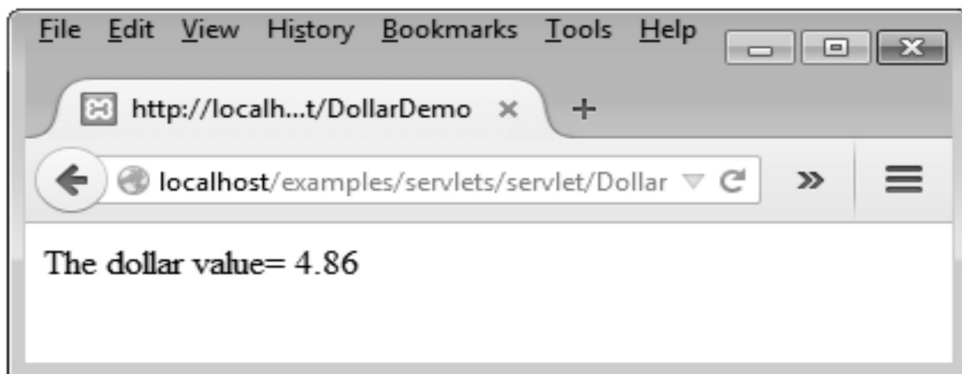
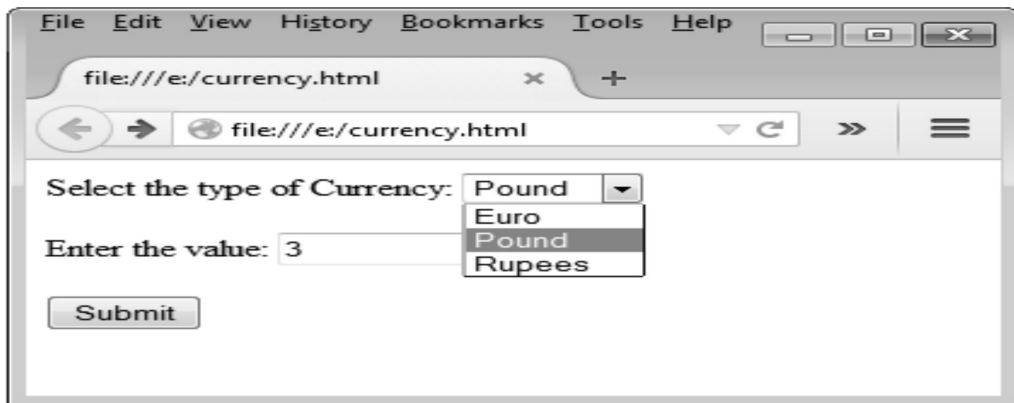
```
import java.io.*;
import java.net.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class DollarDemo extends HttpServlet {
    public void doPost(HttpServletRequest req,HttpServletResponse res)throws
        ServletException,IOException,NullPointerException
    {
        res.setContentType("text/html");
        PrintWriter out=res.getWriter();
        String c=req.getParameter("currency");
        double RupeeValue,EuroValue,PoundValue;
        double DollarValue=0;
        String value=req.getParameter("currencyVal");

        if(c.equals("Rupees"))
        {
            RupeeValue=Double.parseDouble(value);
            DollarValue=RupeeValue/61;
        }
        if(c.equals("Euro"))
        {
            EuroValue=Double.parseDouble(value);
            DollarValue=EuroValue/0.77;
        }
        if(c.equals("Pound"))
        {
```

```
PoundValue=Double.parseDouble(value);
DollarValue=PoundValue*1.62;
}
out.println("<p> The dollar value= "+DollarValue+"</p>");
out.close();
}
}
```

Output



3.8 Sessions

- When you open some application, use it for some time and then close it. This entire scenario is named as **session**.
- **Session state** is a server-based state mechanism that lets web applications store and retrieve objects of any type for each **unique user session**. That is, each browser session has its own session state stored as a serialized file on the server, which is deserialized and loaded into memory as needed for each request. Refer Fig. 3.8.1.

- Because server storage is a finite resource, objects loaded into memory are released when the request completes, making room for other requests and their session objects. This means there can be more active sessions on disk than in memory at any one time.

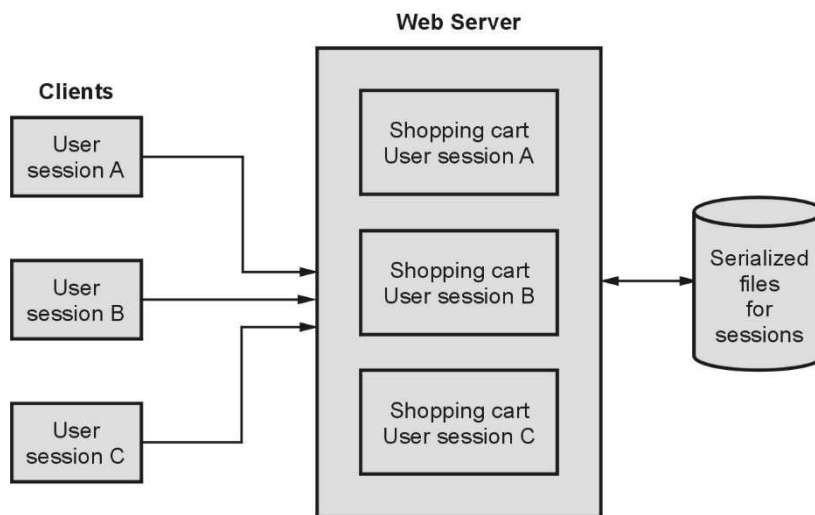


Fig. 3.8.1 Session state

- Sometimes the information about the session is required by the server. This information can be collected during the session. This process is called **session tracking**.
- There are three commonly used techniques used for session tracking and those are,
 - Session ID
 - Cookies
 - URL Rewriting.

3.8.1 Session Management using Session ID

- **HTTP** is a **stateless protocol** in which each request is independent of the previous request. And HTTP is a protocol using which user can interact with the server via web browser and it cannot remember previously held communications but sometimes there is serious need to keep track of previous communication sessions. This can be achieved by **session tracking**.
- The **session tracking technique** is a mechanism by which we can keep track of previous sessions between server and the browser.
- For sending all state information to and fro between browser and server, usually an ID is used. This ID is basically a **session-ID**.

- Thus session-ID is passed between the browser and server while processing the information. This method of keeping track of all the information between server and browser using session-ID is called **session tracking**.
- In servlets, for creating the sessions **getSession()** method can be used. This method returns the object which stores the bindings with the names that use this object. And these bindings can be managed using **getAttribute()**, **setAttribute()**, **removeAttribute()** methods. Actually in session tracking two things are playing an important role -
 1. One is **HttpServletRequest** interface which supports **getSession()** method.
 2. The another class is **HttpSession** class which supports the binding managing functions such as **getAttribute()**, **setAttribute()**, **removeAttribute()** and **getAttributeNames()**.

Let us first discuss the servlet program which returns the number of previous sessions established between client and server. Note that any client can communicate with the server using some web browser only!

Example 3.8.1 *Write a servlet program to keep track of number of times user is visiting the page. Display the count appropriately.*

Solution : Servlet Program(SessionServletDemo)

```
import java.io.*;
import java.util.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class SessionServletDemo extends HttpServlet
{
    public void doGet(HttpServletRequest req,HttpServletResponse res)
    throws ServletException,IOException
    {
        res.setContentType("text/html");
        HttpSession session=req.getSession();
        String heading;
        Integer cnt=(Integer)session.getAttribute("cnt");
        if(cnt==null)
        {
            cnt=new Integer(0);
            heading="Welcome You are accessing the page for the First Time";
        }
        else
        {
            heading="Welcome once again!";
        }
    }
}
```

```
cnt=new Integer(cnt.intValue()+1);
}

session.setAttribute("cnt",cnt);
PrintWriter out=res.getWriter();
out.println("<html>");
out.println("<head>");
out.println("</head>");
out.println("<body>");
out.println("<center>");
out.println("<h1>"+heading);
out.println("<h2> The number of previous access= "+cnt);
out.println("</center>");
out.println("</body>");
out.println("</html>");
}
}
```

Output

To get an output we will use following steps -

1. Compile the Servlet (i.e. a java program) using **javac**. This will generate a class file.
2. Copy this class file to C:\your tomcat directory\examples-folder\WEB-INF\classes
3. Edit the **web.xml** file present at C:\your tomcat directory\examples-folder\WEB-INF like this

```
...
<servlet>
  <servlet-name>SessionServletDemo</servlet-name>
  <servlet-class>SessionServletDemo</servlet-class>
</servlet>
...
<servlet-mapping>
  <servlet-name>SessionServletDemo</servlet-name>
  <url-pattern>/servlet/SessionServletDemo</url-pattern>
</servlet-mapping>
```

4. Start tomcat server.
5. Open the web browser and give the URL for **SessionServletDemo**. Following output can be seen.



If you simply refresh your browser either by pressing F5 or clicking the refresh button then you will get following kind of output.



Program Explanation

1. Our class *SessionServletDemo* is inherited from **HttpServlet** class. And inside the **doGet()** method we can create an object *session* of **HttpSession** class. This method returns the current session for the corresponding request.
2. The **getAttribute(string attribute)** returns the value associated with the *attribute*. We can pass this *attribute* as a parameter to this method. If there is no attribute present then this method returns null. The data type of **getAttribute()** method is **void**.
3. The **setAttribute(string attribute, object value)** is a method which assigns the value passed as the object value to the attribute name. The data type of this method is **void**.

3.9 Cookies

SPPU : Dec.-19, Marks 5

Definition : Cookies are some little information that can be left on your computer by the other computer when we access an internet.

- Generally this information is left on your computer by some advertising agencies on the internet. Using the information stored in the cookies these advertising agencies can keep track of your internet usage, liking of users.
- For the applications like on-line purchase systems once you enter your personal information such as your name or your e-mail ID then it can be remembered by these systems with the help of cookies.
- Sometimes cookies are very much dangerous because by using information from your local disk some malicious data may be passed to you. So it is upto you how to maintain your own privacy and security.

3.9.1 Servlet Support for Cookies

- The **cookie** class is used to create cookies in servlet.
- **Syntax :** The Syntax for constructor for cookies are,
 - Cookie()
 - Cookie(String name, String value)
- Various methods used in Cookie are described in following table

Sr. No.	Method	Purpose
1.	public string getName()	It returns the name of the cookie.
2.	public String getValue()	It returns the value of the cookie.
3.	public string setName()	It sets or changes the name of the cookie.
4.	public String setValue()	It sets or changes the value of the cookie.
5.	public void addCookie(Cookie c)	The cookie is added in the response object of HttpServletResponse interface.
6.	public Cookie[] getCookies()	All the cookies can be returned using this method with the help of HttpServletRequest interface.

- In Servlet following steps are used to support for cookies

Step 1 : Creation of Cookies :

The cookie can be created in Servlet and added to response object using **addCookie** method

```
Cookie cookie=new Cookie("user","XYZ");//creating cookie object  
response.addCookie(cookie);//adding cookie in the response
```

Step 2 : Reading Cookies :

The value from the cookie can be obtained using the `getName()` and `getValue()` methods.

```
Cookie[] my_cookies=req.getCookies();  
int n=my_cookies.length;  
for(int i=0;i<n;i++)  
{  
String name=my_cookies[i].getName();  
String value=my_cookies[i].getValue();  
}
```

3.9.2 Examples

Below is simple HTML form in which a servlet is invoked. This servlet creates a cookie by the name **My_Cookie** and stores the value entered by you in the textbox of HTML form. You can further get the information stored in the cookie by another servlet program **getCookieServlet**. Hence we will write three programs -

1. Our normal HTML script in which some value is entered in the textbox.
2. The servlet program named `mycookieservlet` which will set a cookies and take the value entered by you in the HTML form.
3. The another servlet program named *getCookieServlet* which helps us to view the cookie.

HTML Program

```
<html>  
<head>  
<title>Demo of Cookie</title>  
</head>  
<body>  
<form name="form1" method="post"  
action="http://localhost:8080/examples/servlet/mycookieservlet">  
<h3> Enter the value for my Cookie: </h3>  
<input type="text" name="txt_data" size =30 value=" ">  
<input type="submit" value="Submit">  
</form>  
</body>  
</html>
```

Servlet Program [mycookieservlet.java]

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class mycookieservlet extends HttpServlet
{
    public void doPost(HttpServletRequest req,HttpServletResponse res)
    throws ServletException, IOException
    {
        String txt_data = req.getParameter("txt_data");
        // Create cookie.
        Cookie cookie = new Cookie("My_Cookie", txt_data);
        // Adding cookie to HTTP response.
        res.addCookie(cookie);
        // Write friendly output to browser.
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        out.println("<h2>MyCookie has been set to : ");
        out.println(txt_data);
        out.println("<br><br><br>");
        out.println("This page shows that the cookie has been added");
        out.close();
    }
}
```

We have first created an object of Cookie class using which the cookie can be added using **addCookie()** method.

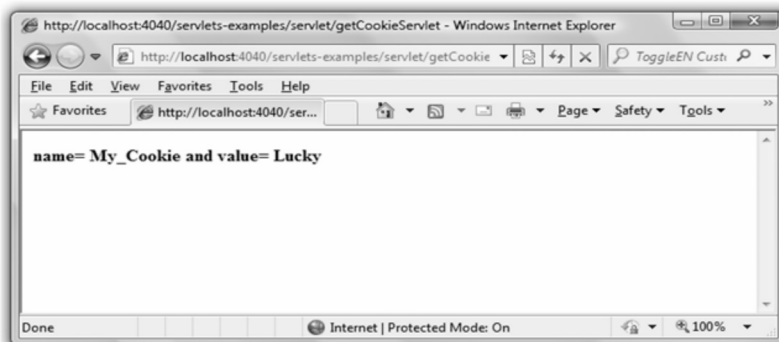
Servlet Program[getcookieservlet.java]

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class getCookieServlet extends HttpServlet
{
    public void doGet(HttpServletRequest req,HttpServletResponse res)
    throws ServletException,IOException
    {
        Cookie[ ] my_cookies=req.getCookies();
        res.setContentType("text/html");
        PrintWriter out=res.getWriter();
        out.println("<b>");
        int n=my_cookies.length;
        for(int i=0;i<n;i++)
        {
```

```
String name=my_cookies[i].getName();
String value=my_cookies[i].getValue();
out.println("name= "+name);
out.println("and value= "+value);
}
out.close();
}
}
```

In the above program using **getName()** and **getValue()** functions we can get the name of the cookie as well as the value of the cookie respectively. The output of all the above given three programs is as given below -

Output



Example 3.9.1 Write a program that allows the user to select a favorite programming language and post the choice to the servlet. The response is a web page in which user can click a link to view list of book recommendations. The cookies previously stored on the client are read by the servlet and form a web page containing the book recommendation. Use servlet, cookies and HTML.

Solution :

Step 1 : Create an HTML page which allows the user to select the favourite programming language. The user can submit his choice by clicking the submit button.

InputForm.html

```
<html>
<body>
<form name="form1" method="post"
action="http://localhost/examples/servlets/servlet/MyProgram">
<h3> Enter the favourite Programming Language: </h3>
<select name="Lang_ch">
    <option value="C++">C++</option>
    <option value="Java">Java</option>
    <option value="PHP">PHP</option>
</select>
<input type="submit" value="Submit">
</form>
</body>
</html>
```

Step 2 : Now create a web servlet program which reads the choice of programming language and set this choice as cookie. The web page containing a hyperlink is created in this servlet program. The code for this Servlet program is as follows -

MyProgram.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class MyProgram extends HttpServlet
{
    public void doPost(HttpServletRequest req,HttpServletResponse res)
    throws ServletException, IOException
    {
        String choice = req.getParameter("Lang_ch");
        // Create cookie.
        Cookie cookie = new Cookie("My_Cookie", choice);
```

```
// Adding cookie to HTTP response.
res.addCookie(cookie);
// Write friendly output to browser with web link
res.setContentType("text/html");
PrintWriter out = res.getWriter();
out.println("<a href='ReadChoice'><p>Click Here to read about your
favorite programming language </p></a>");
out.println("<br><br><br>");
out.println("This page shows that the cookie has been added");
out.close();
}
}
```

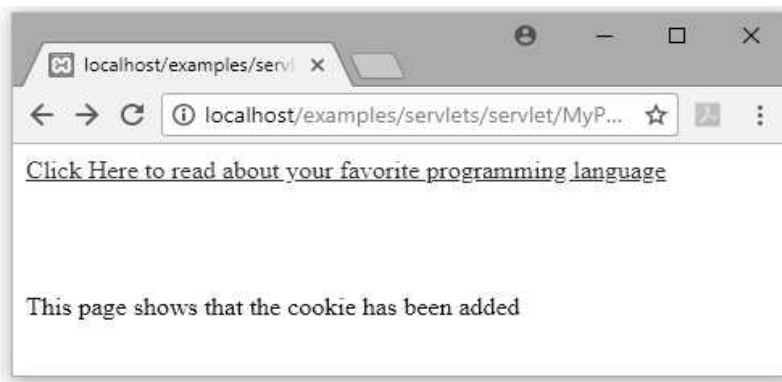
Step 3 : On clicking the hyperlink present in the above servlet, we can read the value of cookie(which is actually the name of the favorite programming language). The following program displays the recommended book for the programming language you have chosen. The code is as follows -

ReadChoice.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class ReadChoice extends HttpServlet
{
    public void doGet(HttpServletRequest req,HttpServletResponse res)
        throws ServletException,IOException
    {
        Cookie[] my_cookies=req.getCookies();
        res.setContentType("text/html");
        PrintWriter out=res.getWriter();
        out.println("<b>");
        int n=my_cookies.length;
        for(int i=0;i<n;i++)
        {
            String name=my_cookies[i].getName();
            String value=my_cookies[i].getValue();
            if(value.equals("PHP"))
            {
                out.println("<b>Programming Language: </b>"+value);
                out.println("<br><b>Book Name: </b>"+ "PHP and MySQL Web
                Development by Luke Welling and Laura Thompson");
            }
            else if(value.equals("Java"))
            {
                out.println("<b>Programming Language: </b>"+value);
            }
        }
    }
}
```

```
        out.println("<br/><b>Book Name: </b>"+"The Complete  
        Reference Java");  
    }  
    else  
    {  
        out.println("<b>Programming Language: </b>" +value);  
        out.println("<br/><b>Book Name: </b>"+"C++  
        Programming Language by Bjarne Stroustrup");  
    }  
}  
out.close();  
}
```

Output



Review Question

1. Explain session and cookies in servlets ? Write any one program to demonstrate session or cookies

SPPU : Dec.-19, Marks 5**3.10 URL Rewriting**

This is more precise technique in which information is embedded into URL. For instance.

```
GET /introduction.html HTTP/1.1  
  
HOST www.mywebpage.com
```

Browser**Requests****Server**

The server then responds by putting **sid=xf1234ad** in response body.

```
HTTP/1.1 200 OK  
<html>  
<a href = introduction.html; sid=xf1234ad>  
    Click here to go to Next page </a>  
    :::  
</html>
```

Browser**Response****Server**

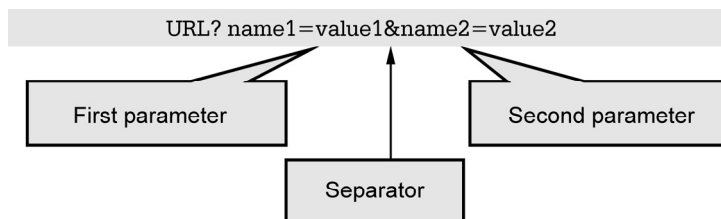
The browser will demand for the required web page by putting the same URL string in the request.

```
GET /introduction.html sid=xf1234ad HTTP/1.1
```

Browser**Request****Server**

When browser makes requests for another page same URL is embedded. There by server will come to know that the request is related to previous page and the work in same session will get co-related.

We can send the name-value pair from one page to another using URL rewriting method. In this method the token of name-value pair can be added at the end of URL. The URL writing occurs in the following format



The parameter sent by the URL is obtained at the server side by **getParameter** method. The syntax of this method is

```
String getParameter(String)
```

Example 3.10.1 Write a servlet that Welcomes the user by his/her name only if the valid password is entered by the user. Make use of URL rewriting method.

Solution :

Step 1 : Create a registration form. I have named it as registration.html. The code for it is as follows :

registration.html

```
<!doctype html>
<html>
<head>
<title> Demo For URL Rewriting </title>
<body>
<form method ="post" action="http://localhost/examples/servlets/servlet/firstpg">
User:<input type ="text" name="user" > <br/>
Password:<input type ="password" name="pwd"> <br/> <br/>
<input type="submit" value="Submit">
</form>
</body>
```

Step 2 : Create a servlet program that reads the user name and password values entered by the user. This servlet also compares the entered password with the valid password. If the user enters valid password, then the request is redirected to another servlet page. Otherwise *wrong password* message will be displayed. The first servlet is,

firstservlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class firstpg extends HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
```

```
response.setContentType("text/html;charset=UTF-8");
PrintWriter out = response.getWriter();
String uname = request.getParameter("user");
String pass = request.getParameter("pwd");
if(pass.equals("password"))
{
    response.sendRedirect("secondpg?user_name="+uname+"");
}
else
    out.println("<i> You have entered wrong password!!!</i>");
}
```

Step 3 : If user enters the valid password, then he/she is redirected to main page. This page welcomes him/her by his/her name. The code for this servlet is as follows -

secondpg.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class secondpg extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        String user = request.getParameter("user_name");
        out.println("<h3>Welcome "+user+" to main page!!!</h3>");
    }
}
```

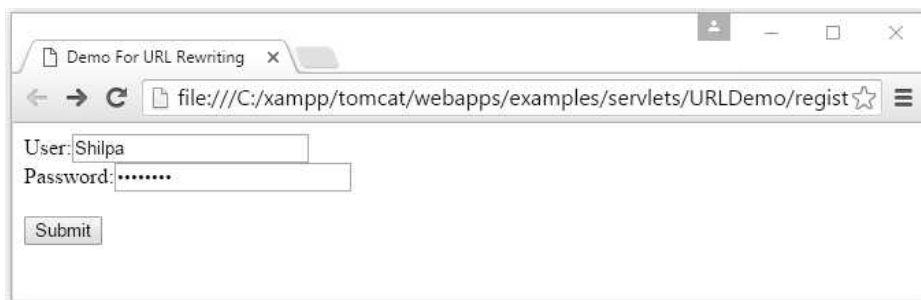
Step 4 : Compile the java files created in step 2 and step 3. Copy the class files to **classes** folder of Tomcat's WEB-INF folder. Modify the web.xml file as follows -

```
<?xml version="1.0" encoding="ISO-8859-1"?>
...
<servlet>
    <servlet-name>firstpg</servlet-name>
    <servlet-class>firstpg</servlet-class>
</servlet>
<servlet>
    <servlet-name>secondpg</servlet-name>
    <servlet-class>secondpg</servlet-class>
</servlet>
```

```
<servlet-mapping>
  <servlet-name>firstpg</servlet-name>
  <url-pattern>/servlets/servlet/firstpg</url-pattern>
</servlet-mapping>
<servlet-mapping>
  <servlet-name>secondpg</servlet-name>
  <url-pattern>/servlets/servlet/secondpg</url-pattern>
</servlet-mapping>
</web-app>
```

Start the Tomcat web server.

Open the suitable web browser and invoke the HTML document as given in Step 1. The output will be,



On clicking submit button we will get the following page on valid password



URL Rewriting with **name** as
"user_name and **value** as "Shilpa"

3.11 Other Servlet Capabilities

In this section we will discuss various methods of **HttpServletRequest** and **HttpServletResponse** class.

3.11.1 Additional `HttpServletRequest` Method

`HttpServletRequest` interface adds the methods that relates to the **HTTP** protocol.

Following table lists various useful methods of `HttpServletRequest`

Method	Description
<code>String getRemoteAddr()</code>	This method returns IP address of the client who is making the request.
<code>String getRemoteHost()</code>	This method returns the name of the client who is making the request.
<code>String getProtocol()</code>	This method returns the name of the protocol, client is using.
<code>String getHeader(String field)</code>	When the field name is given the value for the header is returned.
<code>StringBuffer getRequestURL()</code>	This method returns the String containing the URL which is used to access the servlet. This string does not contain the query string appended to URL.

3.11.2 Additional `HttpServletResponse` Method

- `HttpServletResponse` interface adds the methods that relate to the **HTTP** response. It extends the `ServletResponse` interface.
- Following table lists various useful methods of `HttpServletResponse`

Methods	Description
<code>void addCookie(Cookie cookie)</code>	Adds the specified cookie to the response.
<code>void sendRedirect(String location)</code>	Sends a temporary redirect response to the client using the specified redirect location URL and clears the buffer.
<code>int getStatus()</code>	Gets the current status code of this response.
<code>String getHeader(String name)</code>	Gets the value of the response header with the given name.
<code>void setHeader(String name, String value)</code>	Sets a response header with the given name and value.
<code>void setStatus(int sc)</code>	Sets the status code for this response.
<code>void sendError(int sc, String msg)</code>	Sends an error response to the client using the specified status and clears the buffer.

Example Program

Step 1 :

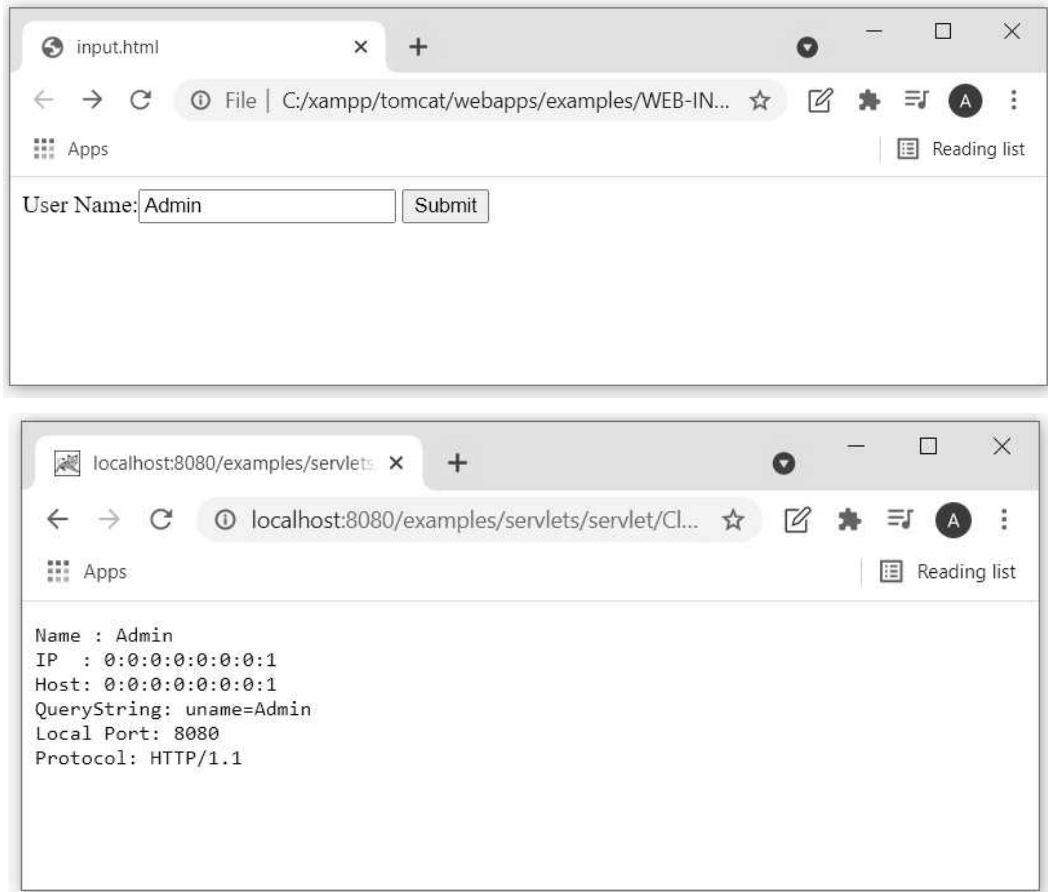
```
<html>
<head>
</head>
<body>
    <form action="http://localhost:8080/examples/servlets/servlet/ClientAddressServlet"
method = "get">
        User Name:<input type="text" name="uname"/>
        <input type="submit" value="Submit"/>
    </form>
</body>
</html>
```

Step 2 :

ClientAddressServlet.java

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
public class ClientAddressServlet extends HttpServlet
{
    public void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException
    {
        // Get client's IP address
        String name = request.getParameter("uname");
        String clientIP = request.getRemoteAddr();
        // Get client's host name
        String clintHost = request.getRemoteHost();
        String queryString = request.getQueryString();
        String protocol = request.getProtocol();
        int localPort = request.getLocalPort();
        response.setContentType("text/plain");
        PrintWriter out = response.getWriter();

        out.println("Name : " + name);
        out.println("IP : " + clientIP);
        out.println("Host: " + clintHost);
        out.println("QueryString: " + queryString);
        out.println("Local Port: " + localPort);
        out.println("Protocol: " + protocol);
    }
}
```

Output

Program Explanation : In above program,

1. In step 1 we have created an HTML form on which there is one text box and one submit button. The name entered in the textbox is identified by the parameter **uname**.
2. In step 2, we have two objects - request and response. These are basically the objects of `HttpRequest` and `HttpResponse` classes.
3. Using **request.getParameter** method we invoke the **uname** parameter. By this, we can access the user name entered by the user in step 1.
4. In the above program, we have used **getRemoteAddr()**, **getRemoteAddr()**, **getProtocol()** methods and display the corresponding information on the web browser.

3.12 Data Storage

- Typical web application contains the HTML documents that often interact with some form of data storage. This data storage can be a file system or a database management system.
- The efficient use of file system or database management system is required by every web application.
- There are two commonly used techniques for data storage - **Java Object Serialization** and **Java Database Connectivity(JDBC)**
- **Serialization in Java** is a mechanism of writing the state of an object into a byte-stream.
- **JDBC** is an API used to assist the Java programs to interact with the database management systems(popularly handled using MySQL)
- Following is an example program that shows **serialization in Java**.

Serialization is the process of writing the state of the object to the byte stream. This technique is useful when we want to store the current state of the object to the File. For serialising the object we need to implement **serializable** interface. This interface does not define any member.

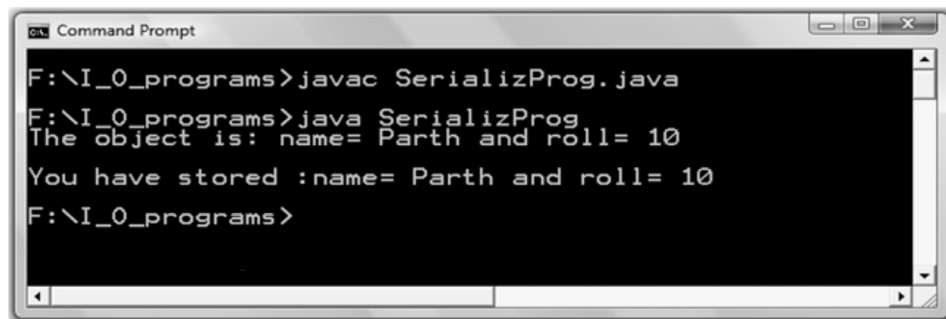
Java Program[SerializProg.java]

```
import java.io.*;
class SerializProg
{
    public static void main(String[] args)throws ClassNotFoundException,IOException
    {
        Myclass myobject=new Myclass("Parth",10);//creating the object
        System.out.println("The object is: "+myobject);//displaying the contents of the object
        ObjectOutputStream o=new ObjectOutputStream(new FileOutputStream("object.txt"));
        o.writeObject(myobject);//writing the object to a file
        o.flush();
        o.close();
        ObjectInputStream i=new ObjectInputStream(new FileInputStream("object.txt"));
        Myclass newobject=(Myclass)(i.readObject());//reading the object from the file
        i.close();
        System.out.println("\nYou have stored :"+newobject);//displaying the object's value
    }
    static class Myclass implements Serializable
    {
        String name;
        int roll;
```

Implements the serializable interface

```
public MyClass(String n,int r)
{
    this.name=n;
    this.roll=r;
}
public String toString()
{
    return "name= "+name+" and roll= "+roll;
}
}
```

Output



Program Explanation

In above program,

1. We have created an object of class named **Myclass**. The members of this class are **name** and **roll number**.
2. From the main function assigns the values to these members. This object is written to the file named **object.txt** using the **writeObject** method.
3. Later on the value of this object can be read using the **readObject** method.
4. For the creation of the object we have to define a class who implements the **Serializable** interface.
5. It is necessary to throw the exceptions **ClassNotFoundException** and **IOException**

3.13 Servlets and Concurrency

- Concurrency allows multiple users to access the system. It also allows system programs to execute concurrently. Also, the web applications implement the concurrency. For instance, when web browser is loading some image, at the same time it also responds to mouse clicks. This type of concurrency is achieved at the client end in client's browser.

- On the server side the situation is different. There can be hundreds or even thousands of users simultaneously accessing a web site.
- Concurrent web servers handle multiple users as follows - It begins with one servlet, executes some instruction, then suspends it for a while and start executing the other servlet. Eventually completes the second servlet and returns back to first servlet to execute the remaining instructions. Refer Fig. 3.13.1.

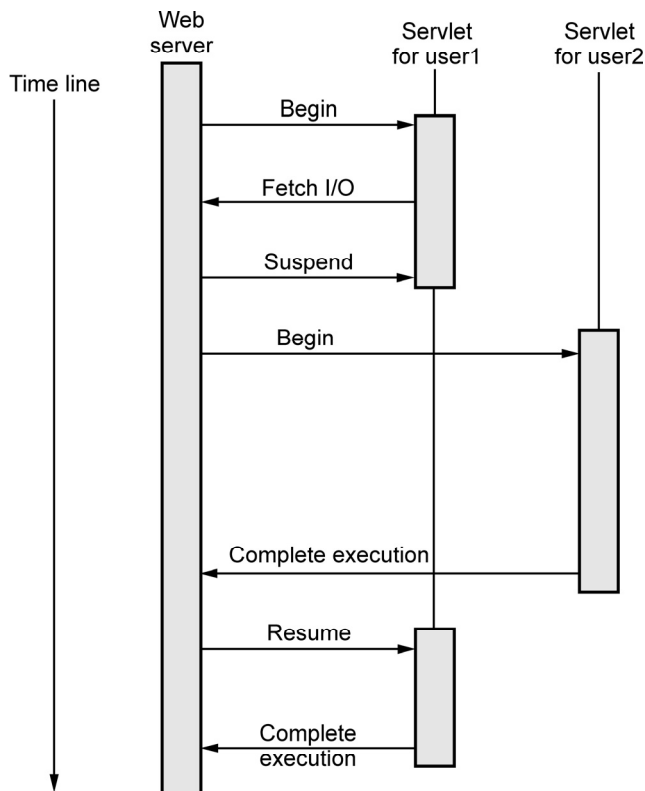


Fig. 3.13.1 Concurrency achieved by web server

Concept of Thread :

- Thread is a light weight process. It is used for concurrent execution of the tasks.
- The Java Virtual Machine maintains a data structure containing the information about each thread. The server creates a thread for each HTTP request. The data structure maintains the information about the state of the thread along with the references to the **HttpServletRequest** and **HttpServletResponse** objects created by the server.
- The Java Virtual Machine also maintains the information whether the particular thread is currently running or not.

- When one thread is suspended then the state of other executing thread is loaded into Java Virtual Machine.
- Following example shows the concurrent execution of the servlet with the help of threads

Step 1 :

```
<HTML>
<BODY>
  <TABLE>
    <TR>
      <TD>
        <a href = "http://localhost:8080/examples/servlets/servlet/SimpleServlet"> servlet1 </a>
      </TD>
    </TR>
    <TR>
      <TD>
        <a href = "http://localhost:8080/examples/servlets/servlet/SimpleServlet"> servlet2 </a>
      </TD>
    </TR>
    <TR>
      <TD>
        <a href = "http://localhost:8080/examples/servlets/servlet/SimpleServlet"> servlet3 </a>
      </TD>
    </TR>
  </TABLE>
</BODY>
</HTML>
```

Step 2 :

```
import java.io.IOException;
import javax.servlet.*;
import javax.servlet.http.*;
import java.math.*;

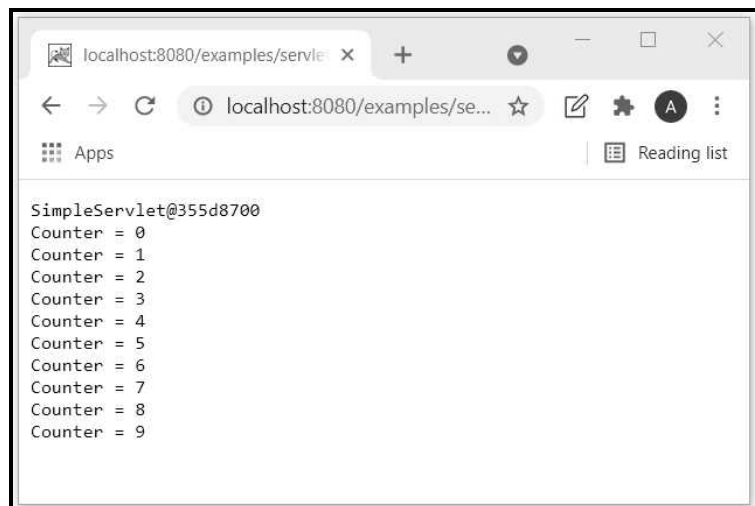
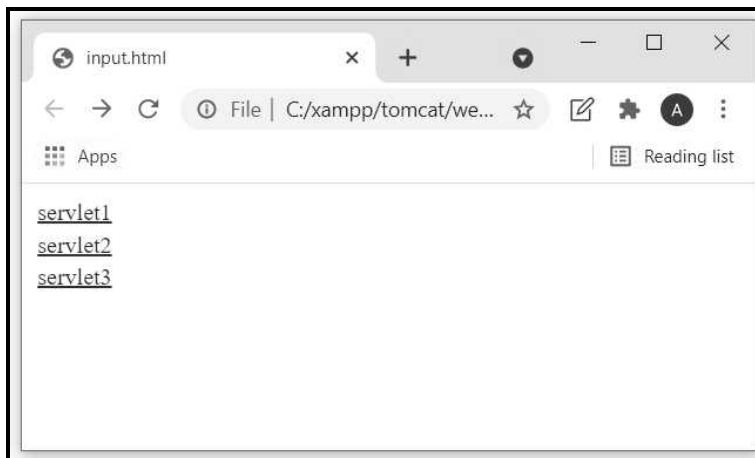
public class SimpleServlet extends HttpServlet
{

    public void doGet(HttpServletRequest req, HttpServletResponse resp)
    throws ServletException, IOException
    {
        doPost(req, resp);
    }

    public void doPost(HttpServletRequest req, HttpServletResponse resp)
    throws ServletException, IOException
```

```
{
    int counter = 0;
    resp.getWriter().println(this);
    for (int i = 0; i < 10; i++)
    {
        resp.getWriter().println("Counter = " + counter);
        try
        {
            Thread.currentThread().sleep((long) Math.random() * 1000);
            counter++;
        }
        catch (InterruptedException exc) { }
    }
}
```

Output



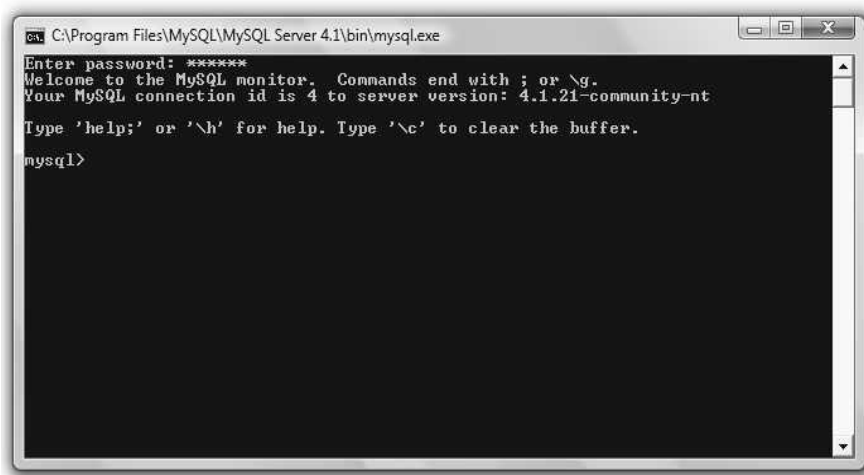
3.14 Databases (MySQL) and Java Servlets

JDBC is a Java API for database access. A servlet can connect to MYSQL and sends the SQL commands to the database and get the required data. Hence we will first get introduced with structured query language SQL.

3.14.1 Structured Query Language using MySQL

The MYSQL is a open source database product which can be downloaded from the web site.

After getting installed on the machine the command prompt window for MYSQL can appear. The screenshot is as follows -



Now let us go through some MYSQL query statements which are required while handling the database.

1. Creating database

```
mysql> CREATE DATABASE mydb;  
Query OK, 1 row affected (0.15 sec)
```

2. Displaying all the databases

```
mysql> SHOW DATABASES;  
+-----+  
| Database |  
+-----+  
| mydb     |  
| mysql    |  
| students |  
| test     |  
+-----+  
4 rows in set (0.06 sec)
```

3. Selecting particular database

```
mysql> USE MYDB;
Database changed
```

4. Creating table

We must create a table inside a database hence it is a common practice to use create table command after USE database command. While creating a table we must specify the table fields.

```
mysql> CREATE TABLE my_table(id INT(4),name VARCHAR(20));
Query OK, 0 rows affected (0.28 sec)
```

Use of Primary Key : The primary key contains the unique value. The primary key column can not contain NUL value. Each table can have only one primary key. Following is a SQL statement used to create PRIMARY KEY.

```
CREATE TABLE student_table(
roll_no INT(4) NOT NULL AUTO_INCREMENT,
name VARCHAR(50) NOT NULL,
address VARACHAR(50) NOT NULL,
PRIMARY KEY(roll_no)
);
```

In above example the roll_no acts as a primary key for the student_table.

5. Displaying a table

After creating the table using SHOW command we can see all the existing tables in the current database.

```
mysql> SHOW TABLES;
+-----+
| Tables_in_mydb |
+-----+
| my_table       |
+-----+
1 row in set (0.00 sec)
```

6. Displaying the table fields

For knowing the various fields of the table we may use following command

```
mysql> DESCRIBE my_table;
+-----+-----+-----+-----+-----+-----+
| Field | Type      | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id    | int(4)    | YES  |     | NULL    |       |
| name  | varchar(20) | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.07 sec)
```

7. Inserting values into the table

We can insert only one complete record at a time. It is as shown below -

```
mysql> INSERT INTO my_table
-> VALUES(1,'SHILPA');
Query OK, 1 row affected (0.05 sec)
```

8. Displaying the contents of the table

```
mysql> SELECT * FROM my_table;
+-----+-----+
| id | name |
+-----+-----+
| 1 | SHILPA |
+-----+-----+
1 row in set (0.06 sec)
```

We can also write SELECT statement for selecting particular row by specifying some condition such as -

```
mysql> SELECT * FROM my_table where id=1;
or
mysql> SELECT * FROM my_table where name='SHILPA';
```

Thus we can insert the rows into the table by repeatedly giving the INSERT command.

If we want to get the records in sorted manner then we use ORDER BY clause

```
mysql> SELECT * FROM my_table;
+-----+-----+
| id | name |
+-----+-----+
| 1 | SHILPA |
| 2 | SUPRIYA |
| 3 | YOGESH |
| 4 | MONIKA |
+-----+-----+
4 rows in set (0.00 sec)
mysql> SELECT * FROM my_table ORDER BY name;
+-----+-----+
| id | name |
+-----+-----+
| 4 | MONIKA |
| 1 | SHILPA |
| 2 | SUPRIYA |
| 3 | YOGESH |
+-----+-----+
4 rows in set (0.00 sec)
```

9. Updating the record

For updating the record in the database following command can be used -

```
mysql> UPDATE my_table
-> SET name='PRIYANKA'
-> WHERE id=4;
Query OK, 1 row affected (0.05 sec)
Rows matched: 1 Changed: 1 Warnings: 0
mysql> SELECT * FROM my_table;
+-----+-----+
| id | name      |
+-----+-----+
| 1 | SHILPA    |
| 2 | SUPRIYA   |
| 3 | YOGESH    |
| 4 | PRIYANKA  |
+-----+-----+
4 rows in set (0.00 sec)
```

10. Deleting record

For deleting particular record from a database

```
mysql> DELETE FROM my_table
-> WHERE id=3;
Query OK, 1 row affected (0.04 sec)
```

Then use SELECT statement for displaying the contents of the table we use following command

```
mysql> SELECT * FROM my_table;
+-----+-----+
| id | name      |
+-----+-----+
| 1 | SHILPA    |
| 2 | SUPRIYA   |
| 4 | PRIYANKA  |
+-----+-----+
3 rows in set (0.00 sec)
```

11. For deleting the table

The table can be deleted using the command

```
mysql> drop table my_table;
```

12. Inner join

This query used to display the result from both the tables when at least one column from both the tables is matching -

For example

Consider table **Customer**

Cust_Id	Name	City
1	Rahul	Bombay
2	Priyanka	Pune
3	Supriya	Bangalore

Consider table **Order**

Order_Id	Cust_Id	Order_Number
10	3	1234
20	1	5678
30	1	8900

Then we can make use of inner join query as follows -

```
SELECT Customer.Name, Customer.city, Order.Order_Number
FROM Customer
INNER JOIN Order
ON Customer.Cust_Id=Order.Order_Id
```

Name	City	Order_Number
Rahul	Bombay	5678
Rahul	Bombay	8900

13. Order by clause

If we want to get the records in sorted manner then we use ORDER BY clause

```
mysql> SELECT * FROM my_table;
```

id	name
1	SHILPA
2	SUPRIYA
3	YOGESH
4	MONIKA

```
mysql> SELECT * FROM my_table ORDER BY name;
```

id	name
4	MONIKA
1	SHILPA
2	SUPRIYA
3	YOGESH

3.14.2 JDBC Perspectives

- JDBC stands for Java DataBase Connectivity.
- JDBC is nothing but an API (i.e. Application Programming Interface).
- It consists of various classes, interfaces, exceptions using which Java application can send SQL statements to a database. The SQL is a Structured Query Language used for accessing the database.
- JDBC is specially used for having connectivity with the RDBMS packages (such as Oracle or MYSQL) using corresponding JDBC driver.

The **JDBC API** is a set of classes, interfaces and exceptions used for establishing connection with data source. This JDBC API is defined in the **java.sql** and **javax.sql** packages. We use following core JDBC classes and interfaces that belong to **java.sql** package.

- **DriverManager** - When Java application needs connection to the database it invokes the **DriverManager** class. This class then loads JDBC **drivers** in the memory. The DriverManager also attempts to open a connection with the desired database. For JSP-MYSQL connectivity we can get DriverManager class as -

```
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
```

- 1) **Connection** - This is an interface which represents connectivity with the data source. The **connection** is used for creating the **Statement** instance.

```
Connection conn=
DriverManager.getConnection("jdbc:mysql://localhost:3306/databaseName",
"userName","password");
```

- 2) **Statement** - This interface is used for representing the SQL statements. Some examples of SQL statements are,
 - SELECT * FROM students_table;
 - UPDATE students_table set name='Neel' where roll_no='1';
 - DELETE from students_table WHERE roll_no='10';

The code for statement is as follows -

```
Statement stmt=conn.createStatement();
stmt.executeQuery("select * from mytab");
```

There are two specialised Statement types: **PreparedStatement** and **CallableStatement**. The **PreparedStatement** represents the precompiled SQL statements.

For instance :

```
SELECT * from students_table WHERE name=?
```

The placeholder represented by ? is used in this type of statement. There are special type setter methods that assign the values to the placeholders before the SQL statement is executed.

CallableStatement is represents the stored procedures. These are similar to **PreparedStatement**. In this type of Statements we can assign the methods for the types of output arguments.

- 3) **ResultSet** - This interface is used to represent the database result set. After using SELECT SQL statement, the information obtained from the database can be displayed using **ResultSet**.
- 4) **SQLException** - For handling SQL exceptions this interface is used.

3.14.2.1 How JDBC Works ?

Following is a way by which JDBC works -

- 1) First of all Java application establishes connection with the data source.
- 2) Then Java application invokes classes and interfaces from JDBC driver for sending queries to the data source.
- 3) The JDBC driver connects to corresponding database and retrieves the result.
- 4) These results are based on SQL statements which are then returned to Java application.
- 5) Java application then uses the retrieved information for further processing.

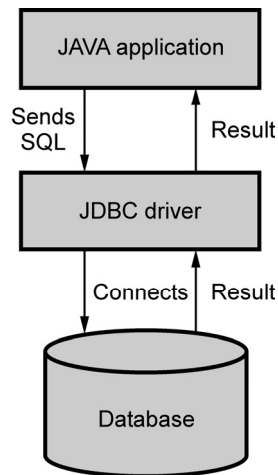


Fig. 3.14.1 Role of JDBC

3.14.2.2 Difference between JDBC and ODBC

The JDBC is similar to the ODBC i.e. Open Database Connectivity. But the ODBC is used for managing any kind of database and JDBC is specially created to support the Java programs.

3.14.2.3 Uses of JDBC

1. JDBC helps the client to store and retrieve the data to the databases.
2. JDBC allows the client to update the databases.

3.14.3 Steps for Connectivity between Servlet and Database

Following steps are used to connect JDBC to MYSQL

Step 1 : Import java.sql.* package in the JDBC program

Following line can be included in your JDBC program at the beginning.

```
import java.sql.*
```

Step 2 : Load JDBC Driver.

The JDBC driver for MYSQL can be loaded using following statement

```
Class.forName("com.mysql.jdbc.Driver");
```

Step 3 : Get connection using the Driver Manager

```
conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/my_database","root","password");
```

Step 4 : Create Statement

```
stmt = conn.createStatement();
```

Step 5 : Execute Query

```
String sql = "SELECT Roll,StudName FROM my_table";  
ResultSet rs = stmt.executeQuery(sql);
```

Step 6 : Display the result

Example 3.14.1 *Explain the JDBC URL with appropriate examples.*

Solution : JDBC URL with examples : The databases are referenced by Java runtime engine using JDBC URL. The syntax of these URLs depends on the database driver you are using.

The list of common drivers along with JDBC URL is,

1. ODBC Driver

Syntax :

```
jdbc:odbc:<data source name>
```

Example :

```
jdbc:odbc:MyDataSource
```

2. MYSQL

Syntax :

```
jdbc:mysql://[host][:port]/[database]
```

Example :

```
jdbc:mysql://localhost:3306/Mydatabase
```

3. ORACLE

Syntax :

```
jdbc:oracle:<drivertype>:<user>/<password>@<database>
```

Example :

```
jdbc:oracle:thin:myuser/mypassword@localhost:mydatabse
```

3.14.4 Implementation

Prerequisite : We need following things for connecting to database using JDBC

1. MySQL is already installed and required database is created.
2. JDK is installed.
3. A database connector mysql-connector-java-X.X.XX-bin must be downloaded and present at the path

```
C:\your_tomcat_directory\common\lib
```

4. Tomcat is running.

For connecting java application with the mysql database, following steps are needed to follow -

1. **Driver class :** The driver class for the mysql database is **com.mysql.jdbc.Driver**. Here we can instantiate the object for JDBC driver by using following statement.

```
Class.forName("com.mysql.jdbc.Driver");
```

2. **Connection URL :** The connection URL for the mysql is

```
databases jdbc:mysql://localhost:3306/my_database
```

Where

jdbc is the API,

mysql is the database,

localhost is the server name on which mysql is running, we may also use IP address, 3306 is the port number

my_database is the database name. (We may use any database, in such case, you need to replace the my_database with your database name.)

Username : The default username for the mysql database is **root**. If you haven't set any user name it could be blank.

Password : Password is given by the user at the time of installing the mysql database.

- The connection URL can be obtained using the method **getConnection**. This method is defined in class named **DriverManager**. Note that this **DriverManager** class is defined in **java.sql.*** package. We can obtain the instance for connection using following statement,

```
conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/my_database","","");
```

Let us understand this with the help of examples.

Example 3.14.2 Write a Servlet program to connect to a MySQL database. Create a student database named StudentDB.

Solution :

1. Create a StudentDB in MySQL using following query

```
create database StudentDB;
```

2. The Servlet program can be written as follows

```
import java.io.*;
import java.sql.Connection;
import java.sql.DriverManager;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

public class ServletDemo extends HttpServlet {
    public void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        try {
            Class.forName("com.mysql.jdbc.Driver");
            Connection con =
DriverManager.getConnection("jdbc:mysql://localhost:3306/StudentDB","","");
            Statement stmt = con.createStatement();
            out.print("You are successfully Connected to Database...");
        }
        catch (SQLException e) {
            throw new ServletException("Could not display records.", e);
        }
        catch (ClassNotFoundException e) {
            throw new ServletException("JDBC Driver not found.", e);
        }
        out.close();
    }
}
```

Example 3.14.3 *Create a students database in any relational database package. The table which you create in this database must have following fields - Roll_No, name, address and phone. The Roll_No must be a primary key. You have to create a servlet at the front end which will perform following operations - 1. Insertion of record 2. Deletion of the record using the primary key 3. Updation on any one field of the record. 4. Display of all the records from the database.*

Solution : We will perform following steps -

Step 1 : First of all we will create a database named **students** and then create a table named **students_table** in this database in MYSQL. This can be done using following SQL commands -

```
mysql>CREATE DATABASE students;
mysql>use students;
mysql>create table students_table(Roll_no INT(4) NOT NULL AUTO_INCREMENT,name
VARCHAR(50) NOT NULL,address VARCHAR(30) NOT NULL,phone VARCHAR(11) NOT
NULL,PRIMARY KEY(Roll_no));
```

Now we have simply created a table without any content within it. Our intention is to insert the records in this table using the front end - servlet program.

Step 2 : The first page of this application will be **Databasedemo.html**. The code for this html document will be as given below -

DatabaseDemo.html

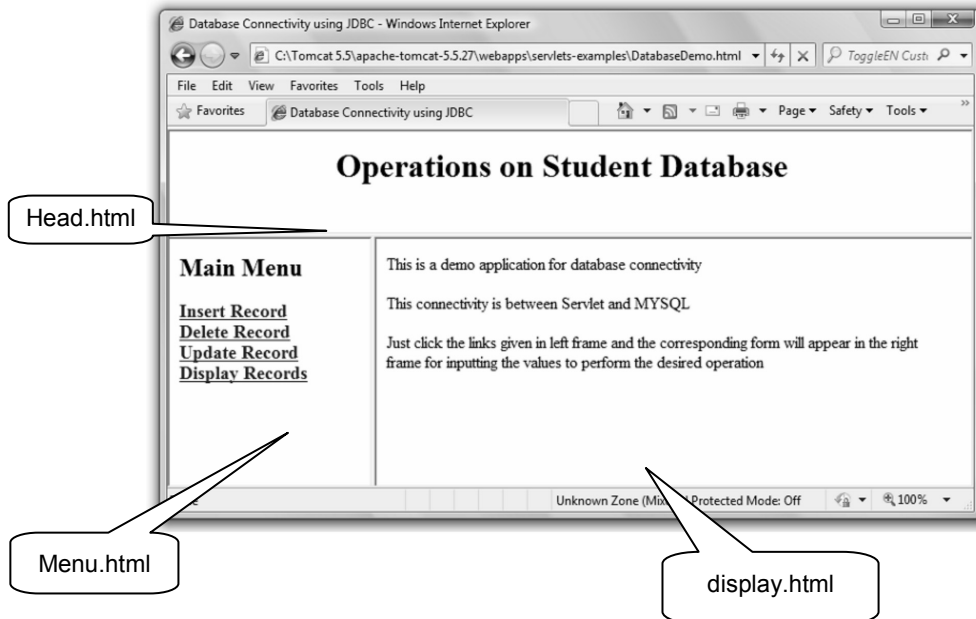
```
<html>
<head>
<title>Database Connectivity using JDBC</title>
</head>
<frameset rows="100,*">
  <framesrc="http://www.localhost.com:4040/servlets-examples/Head.html"
name="top_frame">
  <frameset cols="200,*">
    <framesrc="http://www.localhost.com:4040/servlets-examples/Menu.html"
name="left_frame">
    <framesrc="http://www.localhost.com:4040/servlets-examples/display.html"
name="right_frame">
  </frameset>
</frameset>
</html>
```

The **Databasedemo.html** will simply provide the initial interface for this application. This HTML document consists of three frames. The top frame will simply display the name of the application. The left frame consists of another html page in which the four

links for each operation are specified. Each link will be a separate servlet program communication with the database. The right frame will display the results of each operation.

The output will be as follows -

Step 1 : Open the DatabaseDemo.html web page by giving the path for this html file on the address bar of Internet explorer -



Head.html

```
<html>
<head>
  <title>Database Connectivity</title>
</head>
<body>
  <center><h1>Operations on Student Database</h1></center>
</body>
</html>
```

Menu.html

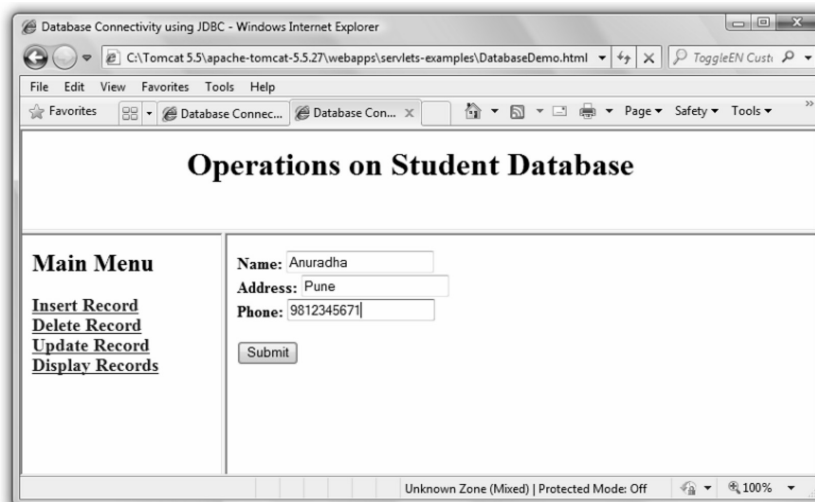
```
<html>
<body>
<h2> Main Menu</h2>
<h3>
<a href="http://www.localhost.com:4040/servlets-examples/servlet/Insert_Query"
target="right_frame">Insert Record</a>
<br/>
```

```

<a href="http://www.localhost.com:4040/servlets-examples/servlet/Delete_Query"
target="right_frame">Delete Record</a>
<br/>
<a href="http://www.localhost.com:4040/servlets-examples/servlet/Update_Query"
target="right_frame">Update Record</a>
<br/>
<a href="http://www.localhost.com:4040/servlets-examples/servlet/ServletDBDemo"
target="right_frame">Display Records</a>
</h3>
</body>
</html>
display.html
<html>
<body>
<p> This is a demo application for database connectivity</p>
<p>This connectivity is between Servlet and MYSQL</p>
<p>Just click the links given in left frame and the corresponding form will
appear in the right frame for inputting the values to perform the desired operation
</p>
</body>
</html>

```

Step 3 : The top frame simply displays the name of the application the left frame displays the various operations that can be performed on the database. There are four sample operations like insertion, deletion, updation and displaying of data from the database. Just click on corresponding links to get the desired operation done. For instance I have clicked on **Insert Record** hyperlink and in the right frame a form appears. Fill up this form and then click the Submit button.



A servlet named **Insert_Query.java** is for inserting a record in the database. It is as follows -

```
Insert_Query.java
import java.io.*;
import java.util.*;
import javax.sql.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class Insert_Query extends HttpServlet
{
    public void service(HttpServletRequest request,HttpServletResponse response)
    throws IOException, ServletException
    {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html>");
        out.println("<head><title>Servlet Database Connectivity</title></head>");
        out.println("<body>");
        out.println("</body></html>");
        // connecting to database
        Connection con = null;
        Statement stmt = null;
        ResultSet rs = null;
        try
        {

            Class.forName("com.mysql.jdbc.Driver");
            con=DriverManager.getConnection("jdbc:mysql://localhost:3306/students","root", "system");
            stmt = con.createStatement();

            out.println("\n\n");
            String Studname=request.getParameter("Sname");
            String Studaddress=request.getParameter("Sadd");
            String Studphone=request.getParameter("Sph");
            if(Studname!=null&&Studaddress!=null&Studphone!=null)
                if(Studname.length()>0)
                    stmt.executeUpdate("insert into students_table(name,address,phone)
                    values('"+Studname+"','"+Studaddress+"','"+Studphone+"')");
```

```
rs=stmt.executeQuery("select * from students_table");
out.println("<html>");
out.println("<body>");
out.println("<form action='Insert_Query' method='post'>");
out.println("<strong>Name: </strong><input name='Sname' type='text'>");
out.println("<br/>");
out.println("<strong>Address: </strong><input name='Sadd' type='text'>");
out.println("<br/>");
out.println("<strong>Phone: </strong><input name='Sph' type='text'>");
out.println("<br/><br/>");
out.println("<input type='submit' value='Submit'>");
out.println("</form>");
    out.println("</body>");
out.println("</html>");

}catch (SQLException e)
{
    throw new ServletException("Servlet Could not display records.", e);
}
catch (ClassNotFoundException e)
{
    throw new ServletException("JDBC Driver not found.", e);
}

finally
{
    try
    {
        if(rs != null)
        {
            rs.close();
            rs = null;
        }
        if(stmt != null)
        {
            stmt.close();
            stmt = null;
        }
        if(con != null)
        {
            con.close();
            con = null;
        }
    }
}
```

```
    catch (SQLException e) {}  
  } //end of finally  
  out.close();  
} //end of service function  
} //end of class
```

Now save this program at the path

C://your_tomcat_directory/webapps/servlets-examples/WEB-INF/classes

Then compile the above code using following command at this path -

```
javac Insert_Query.java
```

If the code is error free then a class file named **Insert_Query.class** gets generated at the above mentioned path.

Now edit the **web.xml** file present in the **../servlets-examples/WEB-INF** directory as follows [Note that at two fragments of code must be added in the web.xml file which are given below]

```
<servlet>  
  <servlet-name>Insert_Query</servlet-name>  
  <servlet-class>Insert_Query</servlet-class>  
</servlet>  
...  
...  
...  
<servlet-mapping>  
  <servlet-name>Insert_Query</servlet-name>  
  <url-pattern>/servlet/Insert_Query</url-pattern>  
</servlet-mapping>
```

Save and close this web.xml file.

The other operations can be performed using following servlet programs. What you need to do is create the following files in Notepad, execute them individually using **javac** command and then edit the **web.xml** file by specifying the name of corresponding servlet.

Delete_Query.java

```
import java.io.*;  
import java.lang.*;  
import java.util.*;  
import javax.sql.*;  
import javax.servlet.*;  
import javax.servlet.http.*;  
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.ResultSet;  
import java.sql.SQLException;  
import java.sql.Statement;
```

```
public class Delete_Query extends HttpServlet
{
    public void service(HttpServletRequest request,HttpServletResponse response)
    throws IOException, ServletException,NumberFormatException
    {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html>");
        out.println("<head><title>Servlet Database Connectivity</title></head>");
        out.println("<body>");
        out.println("</body></html>");
        // connecting to database
        Connection con = null;
        Statement stmt = null;
        ResultSet rs = null;
        try
        {
            Class.forName("com.mysql.jdbc.Driver");
            con = DriverManager.getConnection ("jdbc:mysql://localhost:3306/students","root",
"system");
            stmt = con.createStatement();
            out.println("\n\n");
            String Studroll=request.getParameter("SRoll");
            if(Studroll==null)//this parsing is done to
Studroll="0";//avaiod NumberFormatException
            int n=Integer.parseInt(Studroll);//converting text to int
            stmt.executeUpdate("delete from students_table where Roll_no="+n);
            rs=stmt.executeQuery("select * from students_table");
            out.println("<html>");
            out.println("<body>");
            out.println("<form action='Delete_Query' method='post'>");
            out.println("<strong>Enter Roll Number for deletion : </strong>");
            out.print("<br/>");
            out.print("<input name='SRoll' type='text'>");
            out.println("<br/><br/>");
            out.println("<input type='submit' value='Submit'>");
            out.println("</form>");
            out.println("</body>");
            out.println("</html>");

        }catch (SQLException e)
        {
            throw new ServletException("Servlet Could not display records.", e);
        }
    }
}
```

```
}
catch (ClassNotFoundException e)
{
    throw new ServletException("JDBC Driver not found.", e);
}

finally
{
    try
    {
        if(rs != null)
        {
            rs.close();
            rs = null;
        }
        if(stmt != null)
        {
            stmt.close();
            stmt = null;
        }
        if(con != null)
        {
            con.close();
            con = null;
        }
    }
    catch (SQLException e) {}
} //end of finally
out.close();
} //end of service function
} //end of class
```

Update_Query.java

```
import java.io.*;
import java.lang.*;
import java.util.*;
import javax.sql.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
```

```
public class Update_Query extends HttpServlet
{
    public void service(HttpServletRequest request,HttpServletResponse response)
    throws IOException, ServletException,NumberFormatException
    {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html>");
        out.println("<head><title>Servlet Database Connectivity</title></head>");
        out.println("<body>");
        out.println("</body></html>");
        // connecting to database
        Connection con = null;
        Statement stmt = null;
        ResultSet rs = null;
        try
        {
            Class.forName("com.mysql.jdbc.Driver");
            con=DriverManager.getConnection("jdbc:mysql://localhost:3306/students","root", "system");
            stmt = con.createStatement();
            out.println("\n\n");
            String Studroll=request.getParameter("SRoll");
            String Studadd=request.getParameter("Sadd");
            if(Studroll==null)//this parsing is done to
            Studroll="0";//avoid NumberFormatException
            int n=Integer.parseInt(Studroll);//converting text to int
            stmt.executeUpdate("update students_table set address='"+Studadd+"' where Roll_no="+n);
            rs=stmt.executeQuery("select * from students_table");
            out.println("<html>");
            out.println("<body>");
            out.println("<form action='Update_Query' method='post'>");
            out.println("<strong>Enter Roll Number for updation : </strong>");
            out.print("<br/>");
            out.print("<input name='SRoll' type='text'>");
            out.print("<br/>");
            out.println("<strong>Enter new address : </strong>");
            out.print("<br/>");
            out.print("<input name='Sadd' type='text'>");

            out.println("<br/><br/>");
            out.println("<input type='submit' value='Submit'>");
            out.println("</form>");
            out.println("</body>");
        }
    }
}
```

```
        out.println("</html>");

    }catch (SQLException e)
    {
        throw new ServletException("Servlet Could not display records.", e);
    }
    catch (ClassNotFoundException e)
    {
        throw new ServletException("JDBC Driver not found.", e);
    }

    finally
    {
        try
        {
            if(rs != null)
            {
                rs.close();
                rs = null;
            }
            if(stmt != null)
            {
                stmt.close();
                stmt = null;
            }
            if(con != null)
            {
                con.close();
                con = null;
            }
        }
        catch (SQLException e) {}
    }
    } //end of finally
    out.close();
} //end of service function
} //end of class
```

ServletDBDemo.java

```
import java.io.*;
import java.util.*;
import javax.sql.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.sql.Connection;
import java.sql.DriverManager;
```

```
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class ServletDBDemo extends HttpServlet {
    public void service(HttpServletRequest request,HttpServletResponse response)
        throws IOException, ServletException{
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html>");
        out.println("<head> <title>Servlet Database Connectivity</title> </head>");
        out.println("<body>");
        out.println("</body> </html>");
        // connecting to database
        Connection con = null;
        Statement stmt = null;
        ResultSet rs = null;
        try {
            Class.forName("com.mysql.jdbc.Driver");
            con = DriverManager.getConnection ("jdbc:mysql://localhost:3306/students","root", "system");
            stmt = con.createStatement();
            out.println("Connected to Database...");
            out.println("\n\n");
            rs = stmt.executeQuery("SELECT * FROM students_table");
            // displaying records
            out.print("<center>");
            out.print("<h2>Students Database</h2>");
            out.print("<table border=3>");
            out.print("<th>Roll</th>");
            out.print("<th>Name</th>");
            out.print("<th>Address</th>");
            out.print("<th>Phone</th>");
            while(rs.next())
            {
                out.print("<tr>");
                out.print("<td>");
                out.print(rs.getObject(1).toString());
                out.print("</td>");
                out.print("<td>");
                out.print(rs.getObject(2).toString());
                out.print("</td>");
                out.print("<td>");
                out.print(rs.getObject(3).toString());
                out.print("</td>");
```



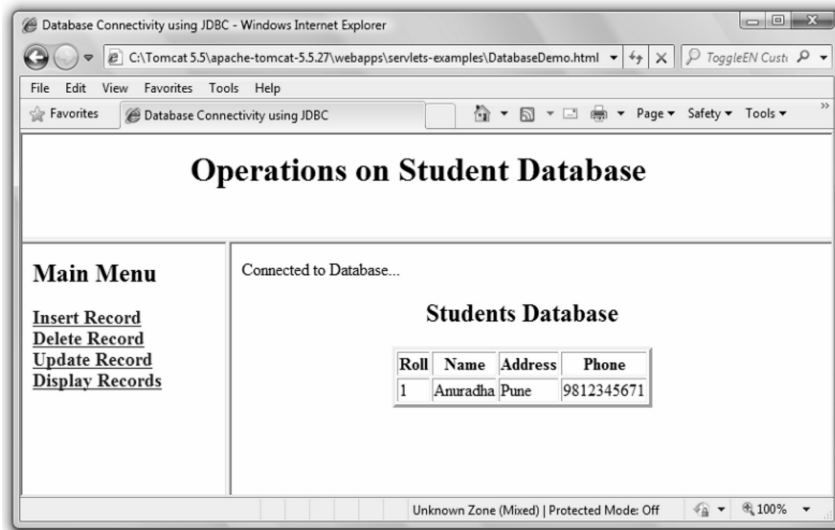
```
        out.print("<td>");
        out.print(rs.getObject(4).toString());
        out.print("</td>");
        out.print("</tr>");
    }
    out.print("</table>");
    out.print("</center>");
} catch (SQLException e) {
    throw new ServletException("Servlet Could not display records.", e);
} catch (ClassNotFoundException e) {
    throw new ServletException("JDBC Driver not found.", e);
} finally {
    try {
        if(rs != null) {
            rs.close();
            rs = null;
        }
        if(stmt != null) {
            stmt.close();
            stmt = null;
        }
        if(con != null) {
            con.close();
            con = null;
        }
    } catch (SQLException e) {}
}
out.close();
}
```

Key Point *All the above servlet code must be compiled in order to generate the class file. Note that all these class files must be present in the folder*

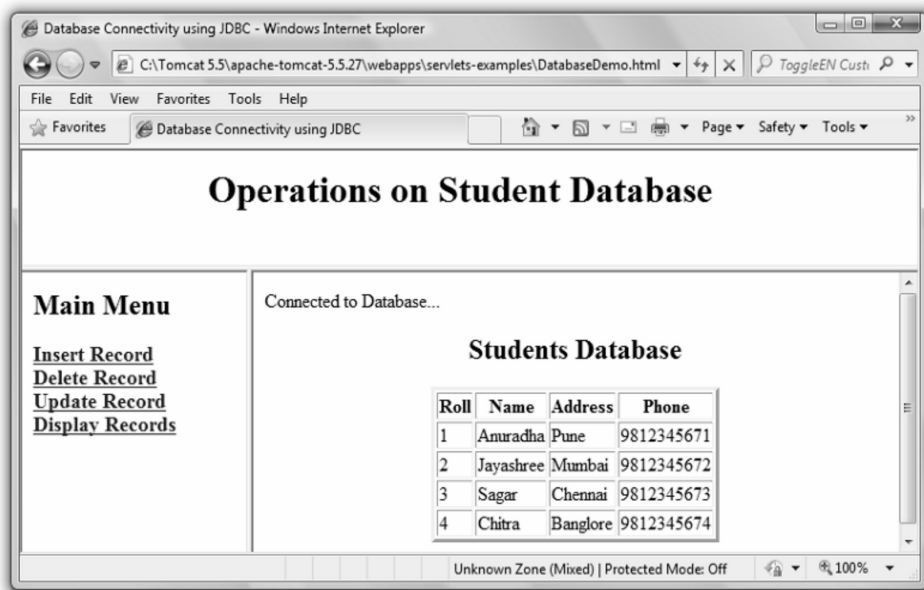
C:\your_tomcat_directory\webapps\servlets-examples\WEB-INF\classes

And the web.xml file must be edited by specifying the class file of each of these servlets.

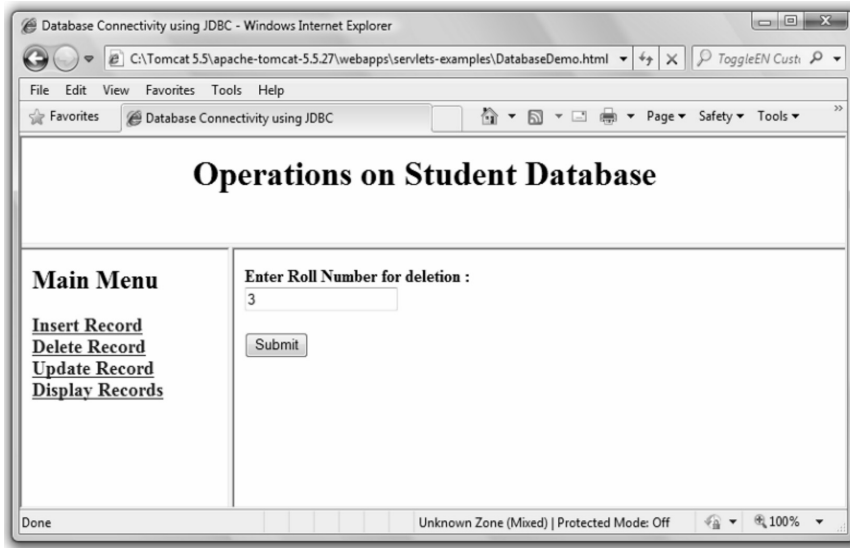
Step 3 : Click on the hyperlink **Display Records** present in the left frame. The contents from the database will be displayed in the right frame. The sample output is -



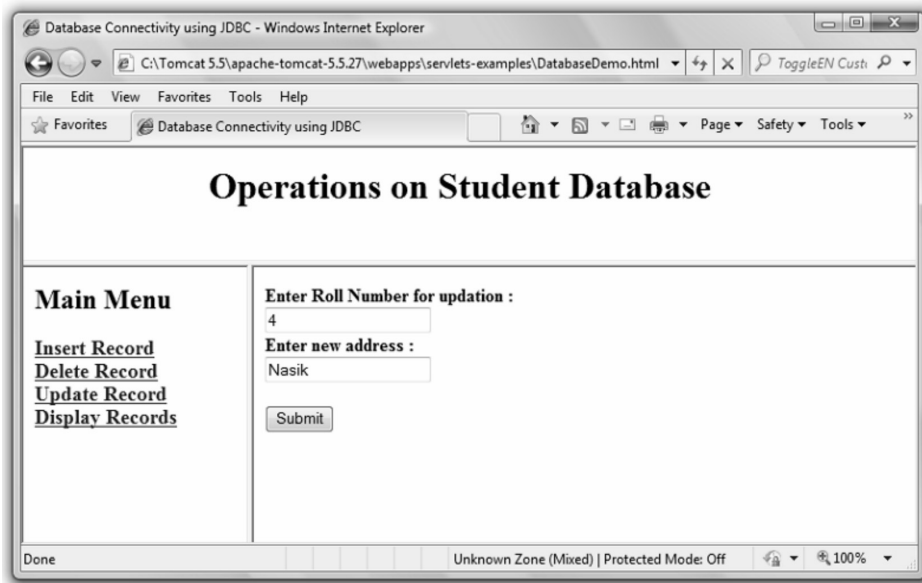
Step 4 : We can insert as many records as we want by clicking the **Insert Record** and filling up the form.



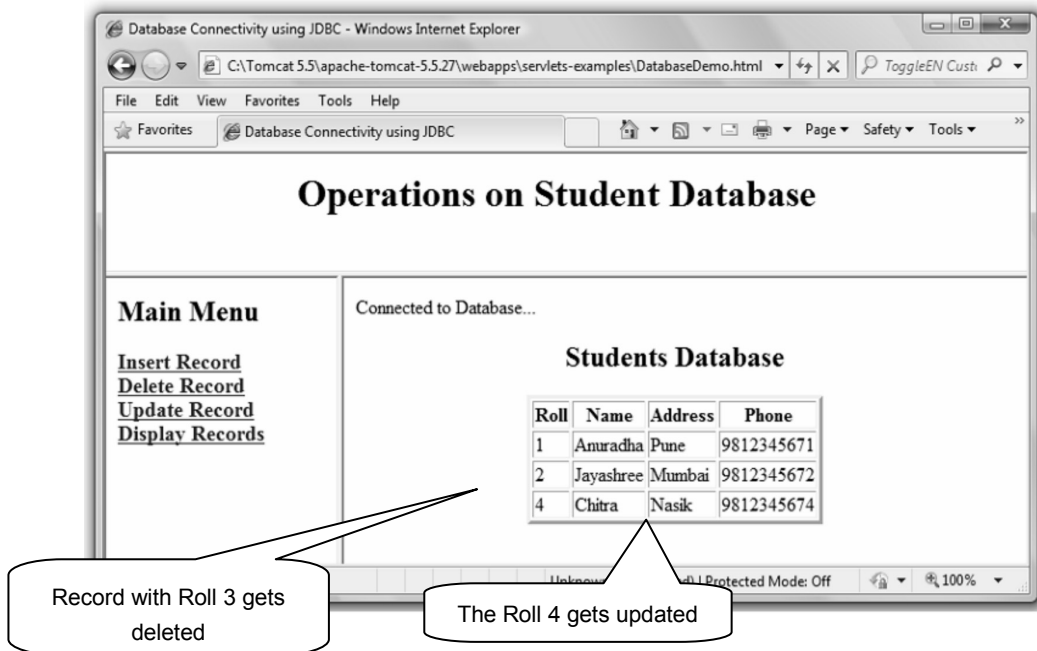
Step 5 : If we want to delete particular record then using the primary key **Roll_No** the corresponding record can be deleted. Just click on the **Delete Record** link present in the left frame and enter the roll number for the record to be deleted. And then click the Submit button. The sample output is follows -



Step 6 : For updating of record click the link **Update Record** present in the left frame. This application allows to update the address using the Roll number. The sample output will be as follows -



Step 7 : You can each time click on the **Display Records** link to see the effects of these operations. The sample output will be -



Example 3.14.4 Consider a database table with the following schema.(PNR No,status). Write a servlet program to display the status, given the PNR number.

Solution : Java Program[PNRDemo.java]

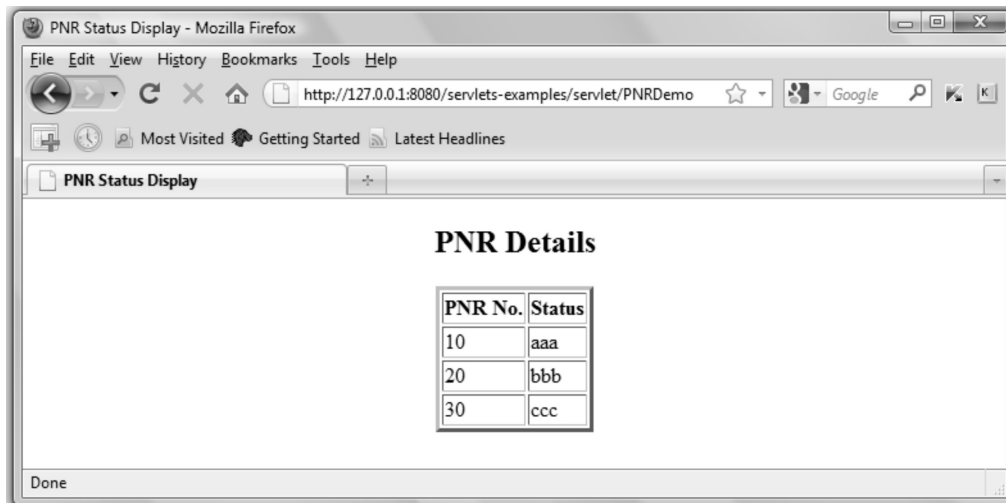
```
import java.io.*;
import java.util.*;
import javax.sql.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class PNRDemo extends HttpServlet
{
    public void service(HttpServletRequest request,HttpServletResponse response)
    throws IOException, ServletException
    {
        // connecting to database
        Connection con = null;
        Statement stmt = null;
        ResultSet rs = null;
        PrintWriter out = response.getWriter();
```

```
try
{
    Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
    con = DriverManager.getConnection ("jdbc:odbc:PNRDemo", "", "");
} catch (ClassNotFoundException e) {e.printStackTrace();}
catch (SQLException e) {e.printStackTrace();}
catch (Exception e) {e.printStackTrace();}
try
{
    stmt = con.createStatement();
    rs = stmt.executeQuery("SELECT * FROM PNR_table");
    // displaying records
    response.setContentType("text/html");
    out.println("<html>");
    out.println("<head><title>PNR Status Display</title></head>");
    out.println("<body>");
    out.print("<center>");
    out.print("<h2>PNR Details </h2>");
    out.print("<table border=3>");
    out.print("<th>PNR No.</th>");
    out.print("<th>Status</th>");
    while(rs.next())
    {
        out.print("<tr>");
        out.print("<td>");
        out.print(rs.getObject(1).toString());
        out.print("</td>");
        out.print("<td>");
        out.print(rs.getObject(2).toString());
        out.print("</td>");
        out.print("</tr>");
    }
    out.print("</table>");
    out.print("</center>");
    out.println("</body></html>");
} catch (SQLException e) { }
finally {
    try {
        if(rs != null) {
            rs.close();
            rs = null;
        }
        if(stmt != null) {
            stmt.close();
            stmt = null;
        }
    }
```

```
}  
if(con != null) {  
    con.close();  
    con = null;  
}  
} catch (SQLException e) {}  
}  
out.close();  
} //end of service function  
} //end of class
```

Output



Example 3.14.5 Assume that a database has a table *Employee* with two columns *EmployeeID* and *Name*. Assume that the administrator user id and password to access the database table are, *scott* and *tiger*. Write a JDBC program that can query and print all the entries in the table *Employee*. Make the database connection using a type2 driver *database.driver* and connection string *jdbc:db:oci*

Solution : The JDBC code for displaying the employees in *Employee* table is as follows -

DisplayEmp.java

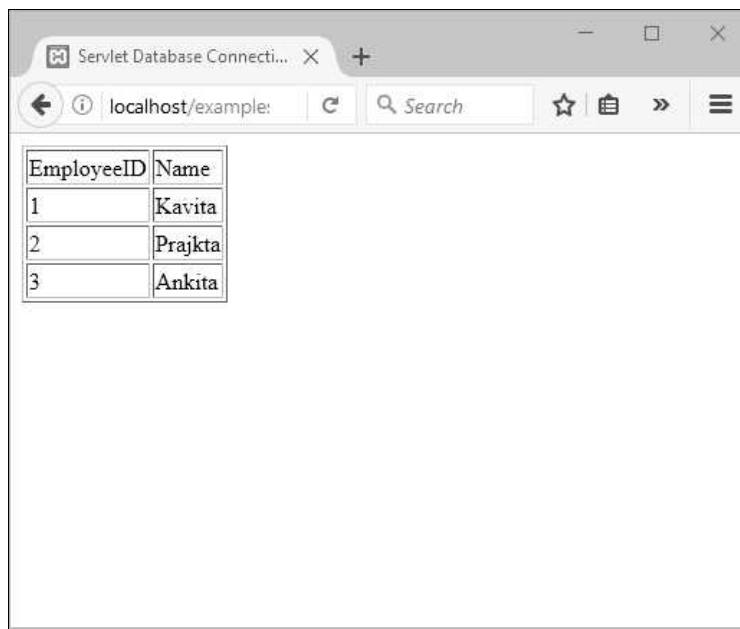
```
import java.io.*;  
import java.util.*;  
import javax.sql.*;  
import javax.servlet.*;  
import javax.servlet.http.*;  
import java.sql.*;  
  
public class DisplayEmp extends HttpServlet
```

```
{
    public void service(HttpServletRequest request, HttpServletResponse response)
        throws IOException, ServletException
    {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html>");
        out.println("<head> <title>Servlet Database Connectivity</title> </head>");
        out.println("<body>");
        // connecting to database
        Connection con = null;
        Statement stmt = null;
        ResultSet rs=null;
        try
        {

            Class.forName("oracle.jdbc.OracleDriver");
            String dbURL = "jdbc:oracle:oci:@my_database";
            Properties properties = new Properties();
            properties.put("user", "scott");
            properties.put("password", "tiger");
            properties.put("defaultRowPrefetch", "20");
            con = DriverManager.getConnection (dbURL,properties);
            stmt = con.createStatement();
            rs=stmt.executeQuery("select * from Employee");
            out.print("<table border=1>");
            out.print("<tr><td>EmployeeID</td><td>Name</td></tr>");
            while(rs.next())
            {
                //Retrieve by column name
                int id = rs.getInt("ID");
                String name = rs.getString("Name");
                out.print("<tr><td>"+id+"</td><td>"+name);
                out.print("</td></tr>");
            }
            out.println("</table></body></html>");
        }catch (SQLException e)
        {
            throw new ServletException("Servlet Could not display records.", e);
        }
        catch (ClassNotFoundException e)
        {
            throw new ServletException("JDBC Driver not found.", e);
        }
    }
}
```

```
finally
{
    try
    {
        if(rs != null)
        {
            rs.close();
            rs = null;
        }
        if(stmt != null)
        {
            stmt.close();
            stmt = null;
        }
        if(con != null)
        {
            con.close();
            con = null;
        }
    }
    catch (SQLException e) {}
} //end of finally
out.close();
} //end of service function
} //end of class
```

Output



The screenshot shows a web browser window with the title 'Servlet Database Connecti...'. The address bar displays 'localhost/example:'. The main content area contains a table with two columns: 'EmployeeID' and 'Name'. The table has three rows of data: (1, Kavita), (2, Prajcta), and (3, Ankita).

EmployeeID	Name
1	Kavita
2	Prajcta
3	Ankita

Part II : XML

3.15 XML Documents and Vocabularies

SPPU : Dec.-18, 19, May-19, Marks 5

- XML stands for eXtensible Markup Language.
- This scripting language is similar to HTML. That means, this scripting language contains various tags. But these tags are not predefined tags, in-fact user can define his own tags.
- Thus HTML is designed for **representation of data** on the web page whereas the XML is designed for **transport or to store data**.

Uses of XML

1. XML is used to display the meta contents i.e. XML describes the content of the document.
2. XML is useful in exchanging data between the applications.
3. The data can be extracted from database and can be used in more than one application. Different applications can perform different tasks on this data.

Advantages of XML

1. XML document is human readable and we can edit any XML document in simple text editors.
2. The XML document is language neutral. That means a Java program can generate an XML document and this document can be parsed by Perl.
3. Every XML document has a tree structure. Hence complex data can be arranged systematically and can be understood in simple manner.
4. XML files are independent of an operating system.

Goals of XML

Following are the goals of XML -

1. User must be able to define and use his own tag. This allows us to restrict the use of the set of tags defined by proprietary vendors.
2. Allow user to build his own tag library based on his web requirement.
3. Allow user to define the formatting rules for the user defined tags.
4. XML must support for storage or transport of data.

Features of XML

Following are some features which are most suitable for creating web related applications.

- XML is EXtensible Markup Language intended for **transport** or storage of the data.
- The most important feature of XML is that user is able to **define and use his own tag**.
- XML contains only data and does not contain any formatting information. Hence document developers **can decide how to display the data**.
- XML permits the document writer to create the tags for any type of information. Due to this virtually any kind of information can be such as mathematical formulae, chemical structures, or some other kind of data can be described using XML.
- Searching sorting, rendering or manipulating the XML document is possible using Extended Stylesheet Language (XSL).
- The XML document can be validated using some external tools.
- Some commonly used web browsers like Internet Explorer and Firefox Mozilla provide support to the tags in XML. Hence XML is not at all vendor specific or browser specific.

Difference between XML and HTML

Sr. No.	HTML	XML
1.	HTML stands for Hypertext Markup Language.	XML stands for eXtensible Markup Language.
2.	HTML is designed to display data with the focus on look and feel of data.	XML is used to transport and store data, with focus on what data is.
3.	HTML is case insensitive.	XML is case sensitive.
4.	HTML has predefined tags.	XML has custom tags can be defined and the tags are invented by the author of the XML document.
5.	As HTML is for displaying the data it is static.	As XML is for carrying the data it is dynamic.
6.	It can not preserve white space.	It can preserve the white space.

Strengths and Limitations

Strengths :

1. XML document is human readable and we can edit any XML document in simple text editors.
2. The XML document is language neutral. That means a Java program can generate an XML document and this document can be parsed by Perl.

3. Every XML document has a tree structure. Hence complex data can be arranged systematically and can be understood in simple manner.
4. XML files are independent of an operating system.

Limitations :

1. XML syntax is verbose and redundant as compared to other text based data transmission formats.
2. Redundancy in syntax of XML causes higher storage and transportation cost when there is large volume of data.
3. XML does not support array.
4. XML document is less readable.
5. Due to verbose nature of XML syntax the XML file size is usually very large.

Review Questions

1. What are the strengths of XML technology ? Also list the limitations of using XML.

SPPU : Dec.-18, Marks 5

2. What are the strengths of XML technology ? Explain the need for XML.

SPPU : May-19, Marks 5

3. Describe the following web technology : XML.

SPPU : Dec.-19, Marks 2**3.16 XML Declaration****Elements and Attributes**

- Various building blocks of XML are -

1. Elements

- The basic entity is **element**. The elements are used for defining the tags. The elements typically consist of opening and closing tag. Mostly only one element is used to define a single tag.

2. Attribute

- The attributes are generally used to specify the values of the element. These are specified within the double quotes.

For example -

```
<flag type="True">
```

- The type attribute of the element flag is having the value True.

3. CDATA

CDATA stands for Character Data. This character data will be parsed by the parser.

4. PCDATA

It stands for Parsed Character Data (i.e. text).

3.16.1 Declaring Elements

- The element type data can be declared using the **ELEMENT**.
- The syntax for defining the ELEMENT is,

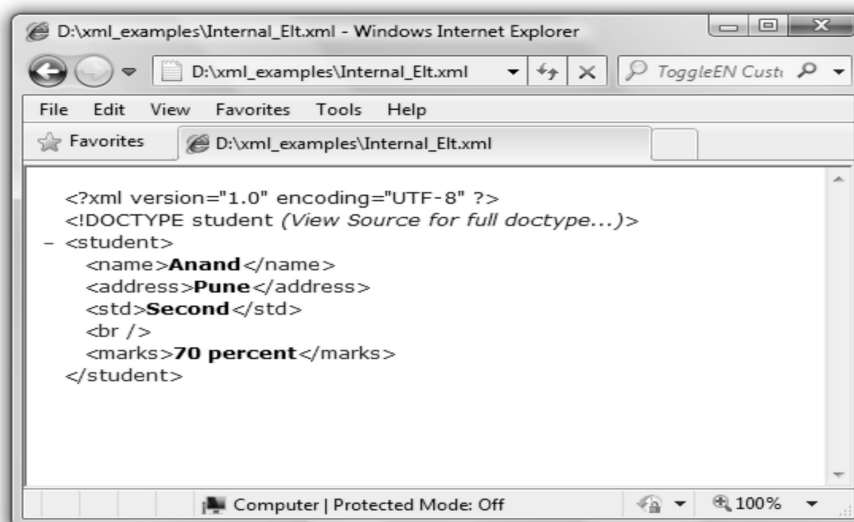
```
<!ELEMENT elt_name (elt1,elt2,...)>
```

XML Document [Internal_Elt.xml]

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE student [
<!ELEMENT student (name,address,std,br,marks)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT address (#PCDATA)>
<!ELEMENT std (#PCDATA)>
<!ELEMENT br EMPTY>
<!ELEMENT marks (#PCDATA)>
]>
<student>
  <name>Anand</name>
  <address>Pune</address>
  <std>Second</std>
  <br />
  <marks>70 percent</marks>
</student>
```

Here we are defining
the DTD internally

Output



- The ELEMENT must be defined in the beginning of the XML file. It must be enclosed within `<!DOCTYPEroot_element ....>`
- There are various ways by which the actual element can be **defined**. It is enlisted in the following table -

Category	Syntax	Description
Empty	<code><!ELEMENT elt_nameEMPTY></code>	Empty elements can be declared.
Character data	<code><!ELEMENT elt_name (#PCDATA)></code>	Character data can be declared.
Any content	<code><!ELEMENT elt_nameANY></code>	Any combination of data can be declared.
Sequence	<code><!ELEMENTelt_name(elt1,elt2,...)></code>	Sequence of elements that must appear in specific order.

- Sometimes we can define the ELEMENT along with the iterator characters. The iterator characters are the special character which are associated with some meaning.
- Some commonly used iterator characters are as given below -

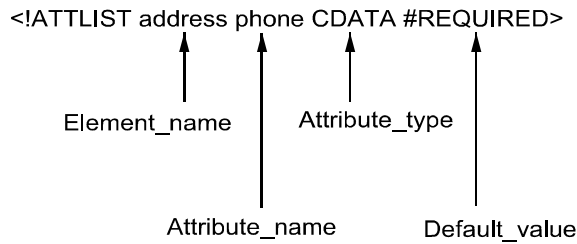
Iterator character	Meaning	Example
?	The preceding choice appears zero or once.	<code><!ELEMENT(marks?)></code> The child element marks can appear for zero or one time.
*	The preceding choice appears for any number of time(zero or more times).	<code><!ELEMENT (person*)></code> The person can appear for any number of times.
+	The preceding choice appears for at least once (one or more times).	<code><!ELEMENT (person+)></code> The person can appear for at least once.

3.16.2 Declaring Attributes

- The attribute type declaration is done using the string ATTLIST.
- The attribute type specifies the type of data which can be used for specifying the attribute. The syntax for specifying the attribute declaration is as follows -

```
<!ATTLISTelement_nameattribute_nameattribute_typedefault_value>
```

- For example



- The XML document containing attributes is as given below -

XML Document [Internal_Attr.xml]

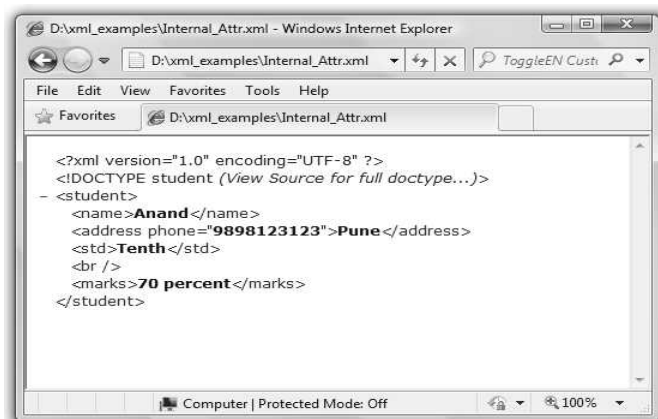
```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE student [
<!ELEMENT student (name,address,std,marks)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT address (#PCDATA)>
<!ELEMENT std (#PCDATA)>
<!ELEMENT marks (#PCDATA)>

<ATTLIST address phone CDATA #REQUIRED>

]>
<student>
  <name>Anand</name>
  <address phone="9898123123">Pune</address>
  <std>Tenth</std>
  <br />
  <marks>70 percent</marks>
</student>
```

Using **ATTLIST** the attribute can be defined.

Output



- Following are some **attribute type** -

Attribute_Type	Syntax	Description
Character data	CDATA	Attribute value is character data.
Enumerated	(str1 str2 ...)	Attribute value must be one from an enumerated list.
Identifier	ID	Attribute value is a unique id.
Identifier reference	IDREF	Attribute value is the id of another element.
Identifier reference list	IDREFS	Attribute value is a list of other ids.
Name token	NMTOKEN	Attribute value is a valid XML name.
Name token list	NMTOKENS	Attribute value is a list of valid XML names.

The default_values are -

Default_Value	Meaning
Value	Default value can be defined. For example - <code><!ATTLIST Index flag="0"></code> In XML document <code><Index flag="1" /></code> If the flag is not assigned with any value then by default it will be 0.
#REQUIRED	Required attribute forces the attribute to be present even if there is no default value.
#IMPLIED	The default value is not provided and attribute can be optionally used.
#FIXED value	The attribute value is fixed and cannot be changed.

3.16.3 Declaring Entities

- The attribute type declaration is done using the string **ENTITY**.
- Inside the XML file at the beginning the entity can be defined using the `<!DOCTYPE ...>` and with the help of string **ENTITY**. Following XML document demonstrates it -

XML Document [EntityDemo.xml]

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE Book[
<!ENTITY writer "Puntambekar">
<!ENTITYnbsp "&#160;">
<!ENTITY title "Web Technologies">
```

```
]>
<Book>&writer;&nbsp;&title;</Book>
```

Output



- Note that we can define the predefined entities ** ** along with the other entity declaration.
- The entity can be used using following syntax.

```
&name_of_entity;
```

- In above script we have used two entities namely **writer** and **title**. For displaying the space within these two entities we have used an entity **nbsp** and for defining it we have used **#160**.

3.17 Rules

- **Rules that must be followed while writing XML -**

Here are some rules that must be followed while writing the XML programs -

1. XML is a **case sensitive**. For example -

```
<hobby>I like drawing</Hobby>or
<Hobby>I like drawing</hobby>
```

is not allowed because the words **hobby** and **Hobby** are treated differently.

2. In XML each **start tag** must have **matching end tag**. For example -

```
<staring> XML is funny to write
<simple> It is simple to implement </simple>
```

That means a closing tag is a must.

3. The elements in XML must be **properly nested**. For example-

```
<one> <two>Hello how are you?</one></two> is wrong but
<one> <two>Hello how are you?</two></one> is correct
```


3.18 XML Namespaces

- Sometimes we need to create two different elements by the same name. The xml document allows us to create **different elements** which are having the **common name**. This technique is known as **namespace**.
- In some web documents it becomes necessary to have the same name for two different elements. Here different elements mean the elements which are intended for different purposes. In such a case support for namespace technique is very much helpful.
- **For example :** Consider the following xml document -

namespacedemo.xml

```
<File-Description>
  <text fname="input.txt">
    <describe>It is a text file</describe>
  </text>
  <text fname="flower.jpg">
    <describe>It is an image file</describe>
  </text>
</File-Description>
```

The above document does not produce any error although the element **text** is used for two different attribute values. The output will be,



3.19 Document Type Definition (DTD)

SPPU : May-18, Marks 5

- The document type definition is used to define the basic building block of any xml document.
- Using DTD we can specify the various elements types, attributes and their relationship with one another.

- Basically DTD is used to specify the set of rules for structuring data in any XML file.
- **For example :** If we want to put some information about some students in XML file then, generally we use tag **student** followed by his/her **name, address, standard and marks**. That means we are actually specifying the manner by which the information should be arranged in the XML file. And for this purpose the document type definition is used.
- Various building blocks of XML are -

1. Elements

The basic entity is **element**. The elements are used for defining the tags. The elements typically consist of opening and closing tag. Mostly only one element is used to define a single tag.

2. Attribute

The attributes are generally used to specify the values of the element. These are specified within the double quotes.

For example -

```
<flag type="True">
```

The type attribute of the element flag is having the value *True*.

3. CDATA

CDATA stands for character data. This character data will be parsed by the parser.

4. PCDATA

It stands for Parsed Character **Data** (i.e. text). Any parsable character data should not contain the markup characters. The markup characters are < or > or &. If we want to use less than, greater than or ampersand characters then make use of < , > or &

- There are two ways by which DTD can be defined.

Internal DTD

Consider following xml document -

XML Document [DTDDemo1.xml]

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE student [
<!ELEMENT student (name,address,std,marks)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT address (#PCDATA)>
<!ELEMENT std (#PCDATA)>
<!ELEMENT marks (#PCDATA)>
]>
<student>
```

Here we are defining the DTD internally

```
<name>Anand</name>
  <address>Pune</address>
  <std>Second</std>
  <marks>70 percent</marks>
</student>
```

Output

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
- <student>
  <name>Anand</name>
  <address>Pune</address>
  <std>Second</std>
  <marks>70 percent</marks>
</student>
```

External DTD

In this type, an external DTD file is created and its name must be specified in the corresponding XML file. Following XML document illustrates the use of external DTD.

Step 1 : Creation of DTD file [*student.dtd*]

Open some suitable text editor or a notepad. Type following code into it -

```
<!ELEMENT student (name,address,std,marks)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT address (#PCDATA)>
<!ELEMENT std (#PCDATA)>
<!ELEMENT marks (#PCDATA)>
```

Now save this file as *student.dtd*

Step 2 : Creation of XML document -

XML Document [*DTDDemo.xml*]

Now create a XML document as follows -

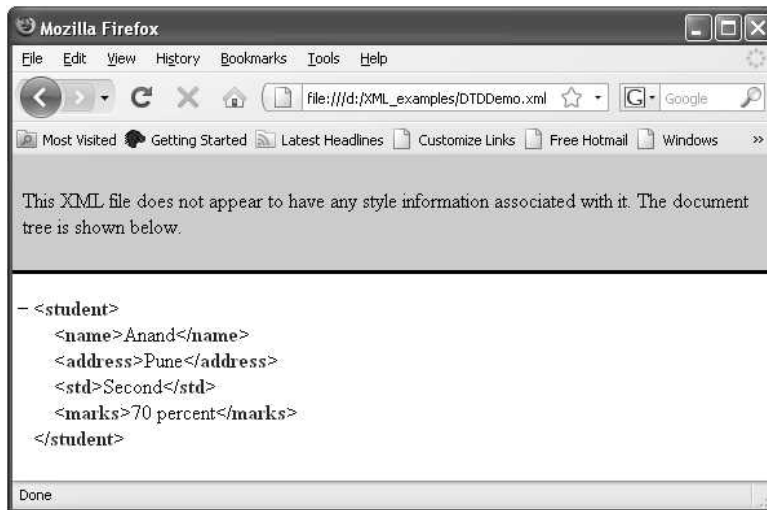
```
<?xml version="1.0"?>
<!DOCTYPE student SYSTEM "student.dtd">
<student>
  <name>Anand</name>
  <address>Pune</address>
  <std>Second</std>
```

The external DTD file is created

```
<marks>70 percent</marks>
</student>
```

Step 3 : Using some web browser open the XML document.

Output



Example 3.19.1 Create a DTD for a catalog of four stroke motorbikes, where each motor bike has the following child elements -make, model, year, color, engine, chassis number and accessories. The engine element has the child elements engine number, number of cylinders, type of fuel. The accessories element has the attributes like disc brake, auto-start and radio, each of which is required and has the possible values yes and no.

Solution :

automobile.dtd

```
<!ELEMENT Catalog (MotorBike)*>
<!ELEMENT MotorBike (make, model, year,color, engine,chassis-num,accessories ) >
<!ELEMENT make (#PCDATA)>
<!ELEMENT model (#PCDATA)>
<!ELEMENT year (#PCDATA)>
<!ELEMENT color (#PCDATA)>
<!ELEMENT engine (engin-num,cylinders-num,fuel-type)>
<!ELEMENT chassis-num (#PCDATA)>
<!ELEMENT accessories (#PCDATA)>
<!ATTLIST accessories disc-brake (yes|no) #REQUIRED
                    auto-start (yes|no) #REQUIRED
                    radio (yes|no) #REQUIRED>
```

Example 3.19.2 Consider a hospital management system. Write a DTD program to consolidate and show the bill to be paid by the inpatients (Assume your own data).

Solution : Patients.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE Patients [
<!ELEMENT Patients (Patient)* >
<!ELEMENT Patient (Name, Address,Test,Date,Doctor,Bill)>
<!ELEMENT Name (#PCDATA)>
<!ELEMENT Address (#PCDATA)>
<!ELEMENT Test (#PCDATA)>
<!ELEMENT Date (#PCDATA)>
<!ELEMENT Doctor (#PCDATA)>
<!ELEMENT Bill (#PCDATA)>
]>
<Patients>
  <Patient>
    <Name>Anuradha</Name>
    <Address>Pune</Address>
    <Test>ECG</Test>
    <Date>1-1-2010</Date>
    <Doctor>Mr.XYZ</Doctor>
    <Bill>Rs.500</Bill>
  </Patient>
  <Patient>
    <Name>Archana</Name>
    <Address>Mumbai</Address>
    <Test>Blood Test</Test>
    <Date>10-3-2010</Date>
    <Doctor>Mr.PQR</Doctor>
    <Bill>Rs.100</Bill>
  </Patient>
</Patients>
```

Merits of DTD

1. DTDs are used to define the structural components of XML document
2. These are relatively simple and compact.
3. DTDs can be defined inline and hence can be embedded directly in the XML document.

Demerits of DTD

1. The DTDs are very basic and hence cannot be much specific for complex documents.

2. The language that DTD uses is not an XML document. Hence various frameworks used by XML cannot be supported by the DTDs.
3. The DTD cannot define the type of data contained within the XML document. Hence using DTD we cannot specify whether the element is numeric, or string or of date type.
4. There are some XML processor which do not understand DTDs.
5. The DTDs are not aware of namespace concept.

Review Question

1. Explain the differences between external and internal DTD.

SPPU : May-18, Marks 5

3.20 Schema

SPPU : May-18, Marks 5

- The XML schemas are used to represent the **structure of XML document**.
- The goal or purpose of XML schema is to define the building blocks of an XML document. These can be used as an alternative to XML DTD.
- The XML schema language is called as **XML Schema Definition (XSD) language**.
- XML schema defines elements, attributes, elements having child elements, order of child elements. It also defines fixed and default values of elements and attributes.
- XML schema also allows the developer to use **data types**.

How to write a simple schema ?

Step 1 : We will first write a simple xsd file in which the desired structure of the XML document is defined. I have named it as *StudentSchema.xsd*

This file contains the complex type with four child simple type elements.

XML Schema [StudentSchema.xsd]

```
<?xml version="1.0"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
    <xs:element name="Student">
        <xs:complexType>
            <xs:sequence>
                <xs:element name="name" type="xs:string"/>
                <xs:element name="address" type="xs:string"/>
                <xs:element name="std" type="xs:string"/>
                <xs:element name="marks" type="xs:string"/>
            </xs:sequence>
        </xs:complexType>
    </xs:element>
</xs:schema>
```

Script Explanation :

1. The **xs** is qualifier used to identify the schema elements and types.
2. The document element of schema is **xs:schema**. The **xs:schema** is the root element. It takes the attribute **xmlns:xs** which has the value **http://www.w3.org/2001/XMLSchema**. This declaration indicates that document should follow the rules of XML schema. The XML schema rules are defined by the W3 recommendation in year 2001.
3. Then comes **xs:element** which is used to define the xml element. In above case the element **Student** is of complex type who have four child elements : *name*, *address*, *std* and *marks*. All these elements are of simple type **string**.

Step 2 : Now develop XML document in which the desired values to the XML elements can be given. I have named this file as **MySchema.xml**

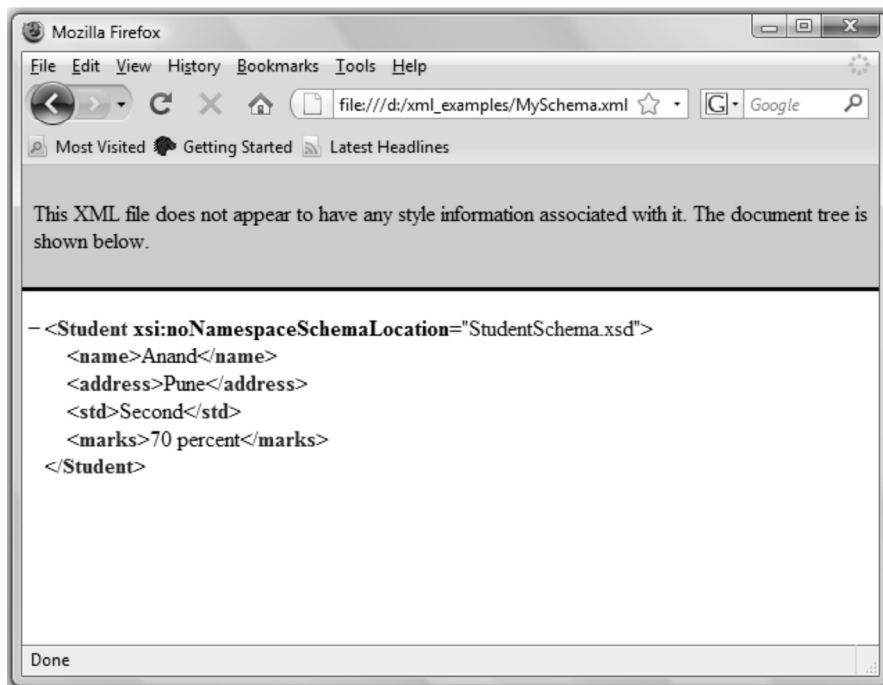
XML Document [MySchema.xml]

```
<?xml version="1.0" encoding="UTF-8"?>
<Student xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="StudentSchema.xsd">
<name>Anand</name>
<address>Pune</address>
<std>Second</std>
<marks>70 percent</marks>
</Student>
```

Program Explanation :

1. In above XML document, the first line is typical in which the version and encoding is specified.
2. There is **xmlns:xsi** attribute of document which indicates that XML document is an instance of XML schema. And it has come from the namespace
"http://www.w3.org/2001/XMLSchema-instance."
3. To tie this XML document with the some Schema definition we use the attribute **xsi:noNamespace SchemaLocation**. The value that can be passed to this attribute is the name of xsd file "StudentSchema.xsd".
4. After this we can define the child elements.

Step 3 : See the output in the browser window as follows -



3.20.1 Data Types

Various data types that can be used to specify the data types of an element are :

- String
- Date
- Numeric
- Boolean.

Let us discuss each of these data types with the help of examples -

1. String data type

The string data type is used to define the **element** containing characters, lines, tabs or white spaces. Following schema definition illustrates this data type.

We can write a schema definition in which the data type of element **Student_Name** is declared as of string type.

XML Schema [stringType.xsd]

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema" >
  <xs:element name="Student_Name" type="xs:string" />
</xs:schema>
```


2. Date data type

For specifying the date we use date data type. The format of this date is yyyy-mm-dd where yyyy denotes the year, mm denotes the month and dd specifies the day. While specifying the date if any entry is single digit then it must be with leading zero. All the entries (i.e. yyyy,mm and dd) must be present.

Here is an illustration.

XML Schema [dateType.dtd]

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
    <xs:element name="Date_of_Birth" type="xs:date"/>
</xs:schema>
```

XML Document [dateTypeDemo.xml]

```
<?xml version="1.0" encoding="UTF-8"?>
<Date_of_Birth xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="dateType.xsd">
2009-09-01
</Date_of_Birth>
```

We can also specify the time by specifying the time as follows -

XML Document

```
<?xml version="1.0" encoding="UTF-8"?>
<Date_of_Birth xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="dateType.xsd">
2009-09-01-01:57
</Date_of_Birth>
```

3. Numeric

If we want to use numeric value for some element then we can use the data types as either **decimal** or **integer**.

XML Schema [decimal Type.xsd]

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
    <xs:element name="My_Marks" type="xs:decimal"/>
</xs:schema>
```

XML Document [decimal Type Demo.xml]

```
<?xml version="1.0" encoding="UTF-8"?>
<My_Marks xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="decimalDemo.xsd">
99.99
</My_Marks>
```

Using decimal data type we can specify the value that may contain some decimal point. We can have another data type called **integer** which is used to specify the numeric values that are without any decimal point.

XML Schema [integer Type.xsd]

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
    <xs:element name="My_Number" type="xs:integer"/>
</xs:schema>
```

4. Boolean

For specifying the true or false values we must use the boolean data type.

XML Schema [booleanType.xsd]

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
    <xs:element name="Flag" type="xs:boolean"/>
</xs:schema>
```

3.20.2 Advantages and Disadvantages of Schema

Advantages of schema

1. The schemas are more specific.
2. The schema provide the support for data types.
3. The schema is aware of namespaces.
4. The XML schema is written in XML itself and has a large number of built in and derived types.
5. The XML schema is the W3C recommendation. Hence it is supported by various XML validator and XML processors.

Disadvantages of schema

1. The XML schema is complex to design and hard to learn.
2. The XML document cannot be if the corresponding schema file is absent.
3. Maintaining the schema for large and complex operations sometimes slows down the processing of XML document.

Advantages of schema over DTD

1. Both the schemas and DTDs are useful for defining structural components of XML. But the DTDs are basic and cannot be much specific for complex operations. On the other hand schemas are more specific.
2. The schemas provide support for defining the type of data. The DTDs do not have this ability. Hence content definition is possible using schema.
3. The schemas are namespace aware and DTDs are not.
4. The XML schema is written in XML itself and has a large number of built in and derived types.
5. The schema is the W3C recommendation. Hence it is supported by various XML validator and XML processors but there are some XML processors which do not support DTD.
6. Large number of web applications can be built using XML schema. On the other hand relatively simple and compact operations can be built using DTD.

Review Question

1. What are XML schemas ? How are they better than DTDs ?

SPPU : May-18, Marks 5

3.21 DOM based XML Processing

What is DOM ?

Document Object Model (DOM) is a set of platform independent and language neutral Application Programming Interface (API) which describes how to access and manipulate the information stored in XML or in HTML documents.

Thus XML DOM is for

- **Loading** the XML document.
- **Accessing** the elements of XML document.
- **Deleting** the elements of XML document.
- **Changing** the elements of XML document.

1. Loading an XML file

We will write a simple script by which any desired XML file can be loaded. Note that these scripts are dependant upon the browser in which these are getting opened. We will write the following script which can be opened in any browser like internet explorer or Mozilla Firefox.

Step 1 : Open some simple text editor like Notepad and type the following script. I have named it as first.html

```
<html>
<!--Simple DOM example for loading some xml file -->
<body>
<script type="text/javascript">
try
{
    xmlDocument=new ActiveXObject("Microsoft.XMLDOM");
}
catch(e)
{
    try
    {
        xmlDocument=document.implementation.createDocument("", "", null);
    }
    catch(e) {alert(e.message)}
}
try
{
    xmlDocument.async=false;
    xmlDocument.load("student.xml");
    document.write("XML Document student.xml is loaded");
}
catch(e) {alert(e.message)}
</script>
</body>
</html>
```

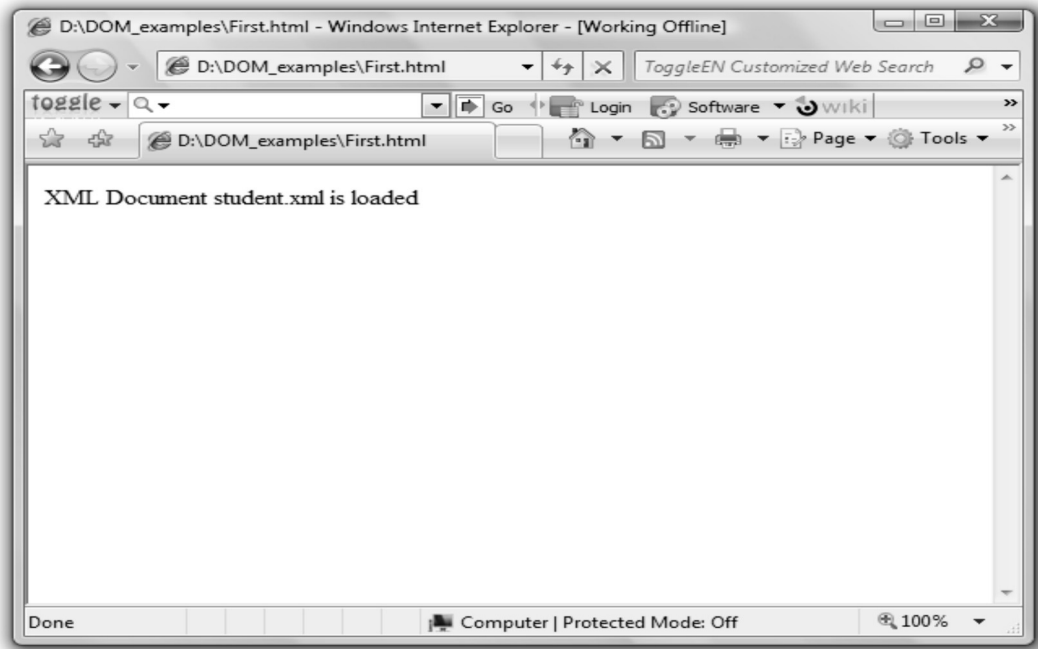
The **load** function is used to load the desired XML file. Note that when we specify some xml file for loading then it must be present at the current path. As an illustration the **student.xml** file is given as below -

XML Document [student.xml]

```
<?xml version="1.0" ?>
<student>
    <Roll_No>10</Roll_No>
    <Personal_Info>
        <Name>Parth</Name>
        <Address>Pune</Address>
        <Phone>1234567890</Phone>
    </Personal_Info>
    <class>Second</class>
```

```
<Subject>Mathematics</Subject>  
<Marks>100</Marks>  
</student>
```

Step 2 : Then open some web browser like Firefox or Internet explorer and the output for the above script can be seen as follows -



Script Explanation

There is an XML parser which reads any XML file. All the modern web browsers like Internet explorer have built in XML parser. The parser converts the XML into javascript accessible object. Hence we have mentioned the script type as -

```
<script type="text/javascript">
```

The parser reads the XML file, stores it into a memory and converts it into the XML DOM object. These objects can be accessible by the **JavaScript**.

Internet Explorer 5 and above have the built in XML parser. Hence in above given script we have written -

```
xmlDocument=new ActiveXObject("Microsoft.XMLDOM");
```

This statement means create an empty XML document object. We have taken this object in the variable **xmlDocument**.

After getting the document loaded fully in the memory the parser will continue for execution of the script, hence we must set -

```
xmlDocument.async=false;
```

Then we should specify the name of the xml file which we want to get loaded.

```
xmlDocument.load("student.xml");
```

Then by using the **document.write** we can display the desired message on the web browser to indicate that the XML file is loaded. Hence we have written

```
document.write("XML Document student.xml is loaded");
```

2. Use of Properties and Methods

Before understanding the properties and methods it is necessary to understand the concept of node tree. In DOM everything from XML is treated as a **node**. Consider a simple XML document,

```
<?xml version="1.0" ?>
<student>
    <Roll_No>10</Roll_No>
    <Personal_Info>
        <Name>Parth</Name>
        <Address>Pune</Address>
        <Phone>1234567890</Phone>
    </Personal_Info>
    <class>Second</class>
    <Subject>Mathematics</Subject>
    <Marks>100</Marks>
</student>
```

We can draw a node tree for this document as follows -

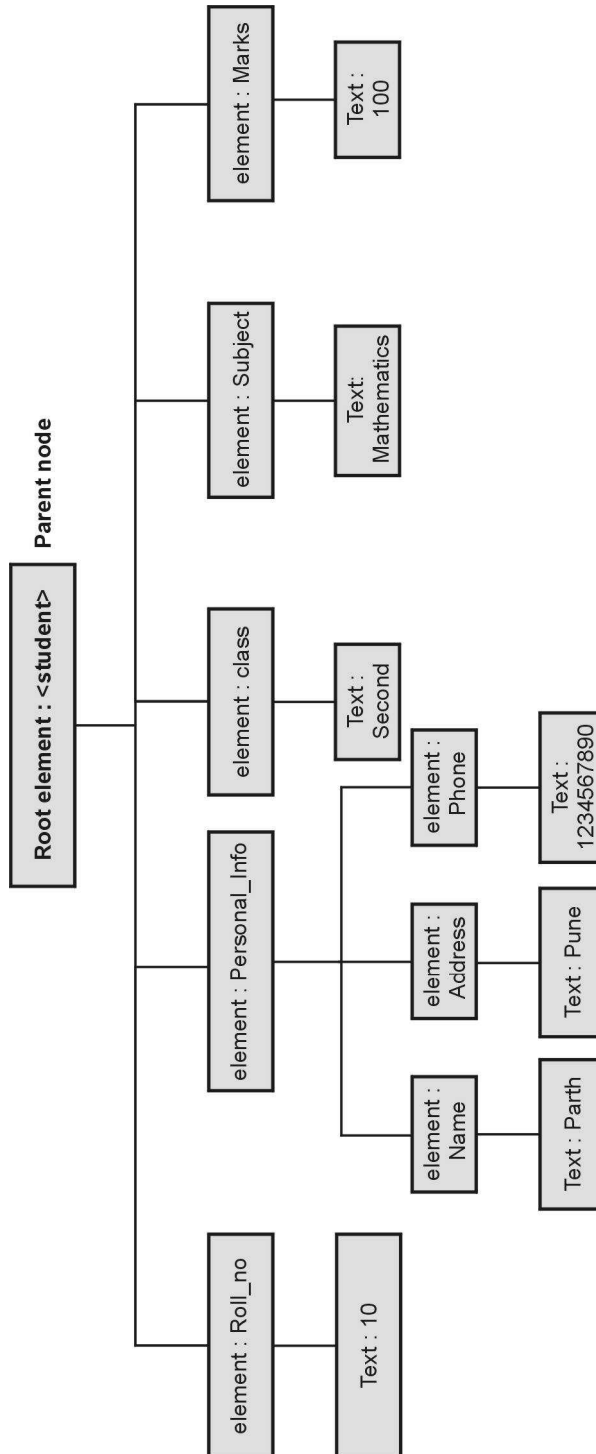


Fig. 3.21.1 Node-tree for student.XML

Various properties are,

Property	Meaning
nodeName	Finding the name of the node.
nodeValue	Obtaining the value of a node.
parentNode	Getting the parent element.
childNodes	Obtaining the child nodes.
attributes	For getting the attributes values.

Various methods are,

Method	Meaning
getElementsByTagName(name)	This method is used to access all the elements that are specified with the tag name.
appendChild(node)	This method is used for inserting a child node.
removeChild(node)	This method is used for removing the child nodes.

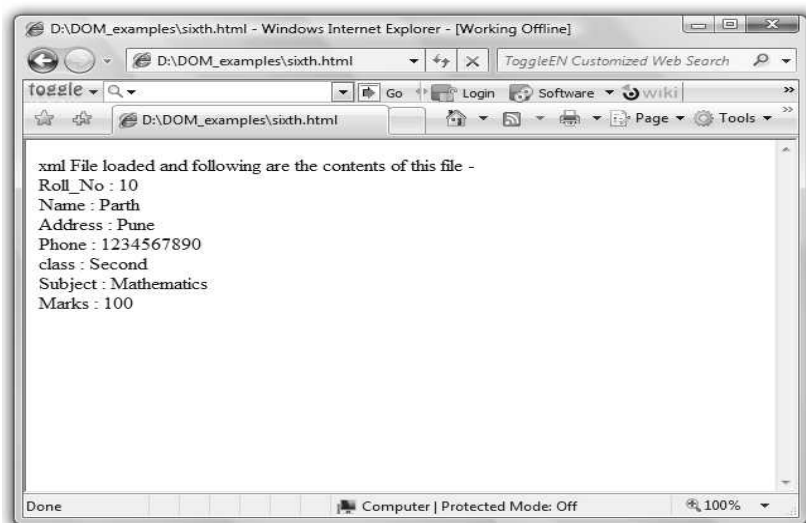
Let us understand the use of properties and methods with the help of following example -

```
<html>
<!-- loading and accessing the contents of xml file -->
<head>
    <script type="text/javascript" src="my_function_file.js"> </script>
</head>
<body>
<script type="text/javascript">
    xmlDocument=My_Function("student.xml");
    document.write("xml File loaded and following are the contents of this file -");
    document.write("<br>");
    document.write("Roll_No :
"+xmlDocument.getElementsByTagName("Roll_No")[0].childNodes[0].nodeValue);
    document.write("<br>");
    document.write("Name :
"+xmlDocument.getElementsByTagName("Name")[0].childNodes[0].nodeValue);
    document.write("<br>");
    document.write("Address :
"+xmlDocument.getElementsByTagName("Address")[0].childNodes[0].nodeValue);
    document.write("<br>");
    document.write("Phone :
"+xmlDocument.getElementsByTagName("Phone")[0].childNodes[0].nodeValue);
    document.write("<br>");
```



```
document.write("class :  
"+xmlDocument.getElementsByTagName("class")[0].childNodes[0].nodeValue);  
document.write("</br>");  
document.write("Subject :  
"+xmlDocument.getElementsByTagName("Subject")[0].childNodes[0].nodeValue);  
document.write("</br>");  
document.write("Marks :  
"+xmlDocument.getElementsByTagName("Marks")[0].childNodes[0].nodeValue);  
document.write("</br>");  
</script>  
</body>  
</html>
```

Output



Script Explanation

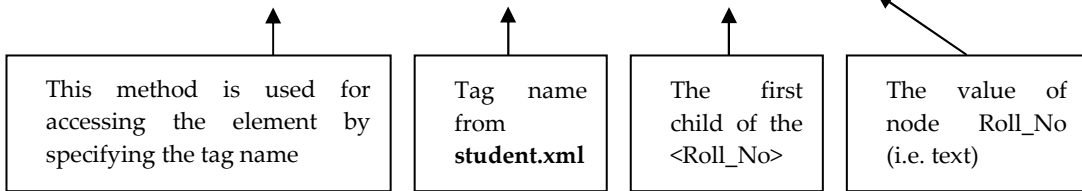
In above given script, we have specified an external file **my_function_file.js** in the head section. This is a file in which the function **My_Function** is defined. This function performs the task of loading an XML document **student.xml**

We will first obtain a document object using the statement,

```
xmlDocument=My_Function("student.xml");
```

Note that **xmlDocument** is a document object referring the document **student.xml**. Using this object we can use the properties and methods. The method **getElementsByTagName** is used and to this method the parameter will be a tag names such as "Roll_No", "Name" "class" and so on.

```
xmlDocument.getElementsByTagName("Roll_No")[0].childNodes[0].nodeValue);
```



Hence as an output we can see the value of every element. That means the contents of **student.xml** file will be displayed.

3.22 Transforming XML Documents

SPPU : May-19, March-20, Marks 5

- The XSLT stands for **XSL Transformations** and XSL stands for **eXtensible Stylesheet Language**.
- An XSLT is a W3C recommendation.
- The XSLT is used for defining the XML document transformations and presentations. Hence XSL decides how an XML document should look on the web browser.
- XSL consists of three parts :
 1. XSLT is a language for **transforming XML documents**.
 2. XPath is a language for **navigating in XML documents**. In other words we can reach to any node of XML document using XPath.
 3. XSL-FO is a language for formatting XML documents for **displaying it in desired manner**.

3.22.1 Transforming XML into XSLT

- Using XSLT we can decide the way by which we want the element to get displayed.
- XSLT uses **XPath** to find information in an XML document. XPath is used to navigate through elements and attributes in XML documents. We can sort these elements, hide or display particular element and can apply some decision making logic on these elements.
- XSLT processors **take two input documents** one is **XML document** and another is **XSLT document**. The XSLT document is nothing but a program and XML document is nothing but the input data, thus this program works on the XML input data. Then some or whole part of the XML document is selected, modified and merged with XSLT program document in order to produce another document.
- This newly produced document is provided as input to the XSLT processor which in turn produce another document called the **XSL document**.

- The XSL document is used along with the application so that particular application can be displayed on the web browser in some desired manner.

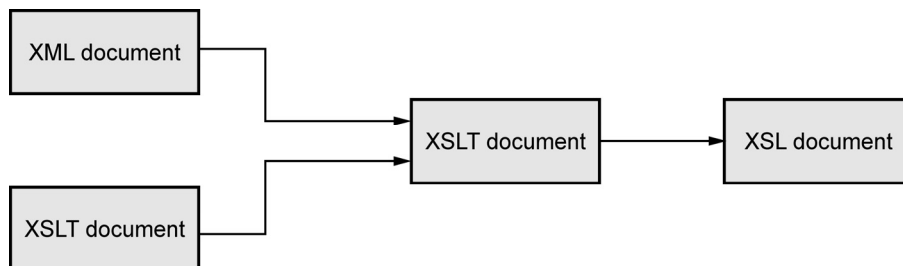


Fig. 3.22.1 Processing of XSLT document

- The XSLT document makes use of **templates** using which particular code can be described in XML document.
- XML document and then particular code in XSLT is executed when the match is found.
- XSLT processor processes the XML document sequentially by reading each line one by one.
- The XSLT model can be described as **template driven** or **data driven** model.

3.22.2 XSL Elements

1. The <xsl:template>

- The <xsl:template> is for building the templates.
- The **match** attribute is associated with the template element. The match attribute can also be used to define a template for the entire XML document. The **match="/"** defines the whole document.
- In the following example we store the XML elements in the tabular form using the <xsl:template> and <match>

Step 1 : Write a simple XML document. One sample example is as below –

Open Notepad and write the following code. Save it by the name **SimpleXml.xml**

XML Document[SimpleXml.xml]

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="SimpleXml.xsl"?>
<Student>
<Person-Details>
  <name>Anand</name>
  <address>Pune</address>
  <std>Second</std>
```

We have defined this file in step 2

```
<marks>70 percent</marks>
</Person-Details>
<Person-Details>
  <name>Anuradha</name>
  <address>Chennai</address>
  <std>Second</std>
  <marks>90 percent</marks>
</Person-Details>
<Person-Details>
  <name>Archana</name>
  <address>Mumbai</address>
  <std>Forth</std>
  <marks>80 percent</marks>
</Person-Details>
<Person-Details>
  <name>Monika</name>
  <address>Delhi</address>
  <std>Tenth</std>
  <marks>77 percent</marks>
</Person-Details>
</Student>
```

Step 2 : Now create a XSL stylesheet. Note that this document must have file name extension as **.xsl**. Open notepad and save following code in it.

XSLT Document [SimpleXml.xsl]

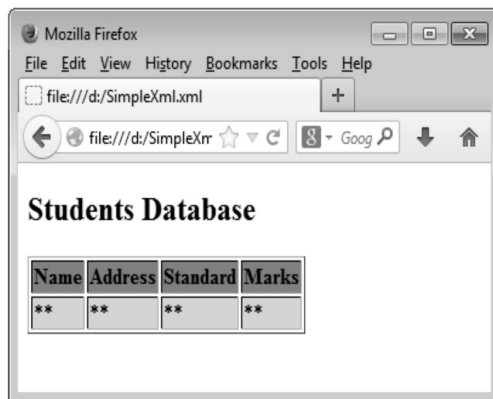
```
<?xml version="1.0" encoding="ISO-8859-1"?>
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:template match="/">
    <html>
    <body>
    <h2>Students Database</h2>
    <table border="1">
    <tr bgcolor="gray">
    <th>Name</th>
    <th>Address</th>
    <th>Standard</th>
    <th>Marks</th>
    </tr>
    <tr bgcolor="pink">
    <td>*</td>
    <td>*</td>
    <td>*</td>
```

```

<td>**</td>
</tr>
</table>
</body>
</html>
</xsl:template>
</xsl:stylesheet>

```

Step 3 : Open the web browser and the output will be as follows -



2. The <xsl:value-of>

The **value-of** is used to extract the value of xml element and add it to output stream of the XSL transformation. Following example illustrates this :

Step 1 : Keep the same xml[**SimpleXml.xml**] file which you have created in section 3.22.2 (1) Step 1.

Step 2 : Create the xsl file as follows -

```

<?xml version="1.0" encoding="ISO-8859-1"?>
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
<xsl:template match="/">
<html>
<body>
<h2>Students Database</h2>
<table border="1">
<tr bgcolor="gray">
<th>Name</th>
<th>Address</th>
<th>Standard</th>
<th>Marks</th>
</tr>

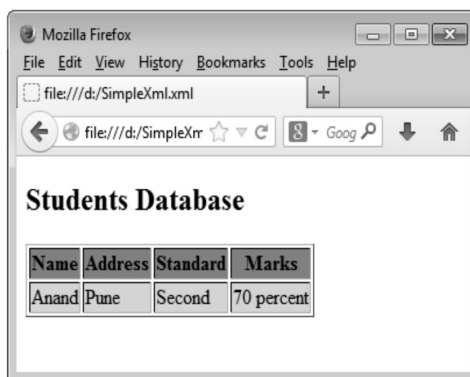
```

```

<tr bgcolor="pink">
<td><xsl:value-of select="Student/Person-Details/name"/></td>
<td><xsl:value-of select="Student/Person-Details/address"/></td>
<td><xsl:value-of select="Student/Person-Details/std"/></td>
<td><xsl:value-of select="Student/Person-Details/marks"/></td>
</tr>
</table>
</body>
</html>
</xsl:template>
</xsl:stylesheet>

```

Step 3 : Open the web browser and the output will be as follows -



3. The <xsl:for-each>

The **for-each** element allows to traverse through each element of the node

Step 1 : Keep the same xml[**SimpleXml.xml**] file which you have created in section 3.22.2(1) Step 1.

Step 2 : Now create .xsl document illustrating <xsl:for-each>. It is as follows -

```

<?xml version="1.0" encoding="ISO-8859-1"?>
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
<xsl:template match="/">
<html>
<body>
<h2>Students Database</h2>
<table border="1">
<tr bgcolor="gray">
<th>Name</th>
<th>Address</th>
<th>Standard</th>

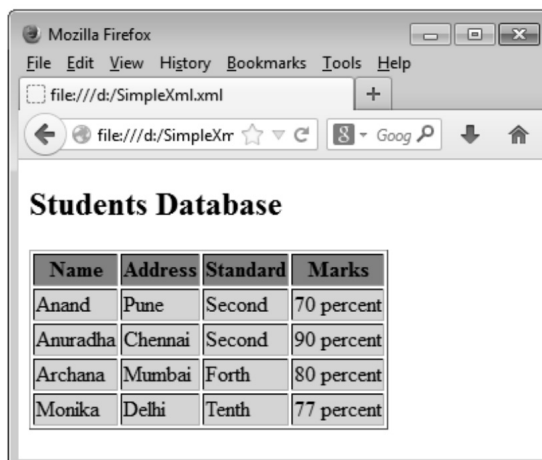
```

```

<th>Marks</th>
</tr>
<xsl:for-each select="Student/Person-Details" >
<tr bgcolor="pink">
<td><xsl:value-of select="name"/> </td>
<td><xsl:value-of select="address"/> </td>
<td><xsl:value-of select="std"/> </td>
<td><xsl:value-of select="marks"/> </td>
</tr>
</xsl:for-each>
</table>
</body>
</html>
</xsl:template>
</xsl:stylesheet>

```

Step 3 : Open the web browser and the output will be as follows -



4. The <xsl:sort>

We can also display the sorted data using <xsl:sort>.

Step 1 : Keep the same xml[SimpleXml.xml] file which you have created in section 3.22.2 (1) Step 1.

Step 2 : Create the .xsl file as follows -

SimpleXml.xsl

```

<?xml version="1.0" encoding="ISO-8859-1"?>
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
<xsl:template match="/">

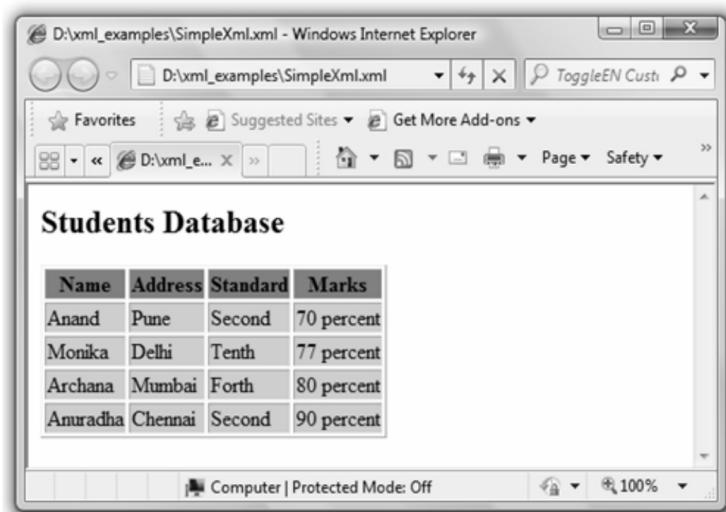
```

```

<html>
<body>
<h2>Students Database</h2>
<table border="1">
<tr bgcolor="gray">
<th>Name</th>
<th>Address</th>
<th>Standard</th>
<th>Marks</th>
</tr>
<xsl:for-each select="Student/Person-Details">
<xsl:sort select="marks"/>
<tr bgcolor="pink">
<td><xsl:value-of select="name"/></td>
<td><xsl:value-of select="address"/></td>
<td><xsl:value-of select="std"/></td>
<td><xsl:value-of select="marks"/></td>
</tr>
</xsl:for-each>
</table>
</body>
</html>
</xsl:template>
</xsl:stylesheet>

```

Output



5. The <xsl:if>

For selection specific record based on some condition the <xsl:if> is used.

Step 1 : Keep the same xml[**SimpleXml.xml**] file which you have created in section 3.22.2(1) Step 1.

Step 2 : Create the .xsl file as follows -

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
<xsl:template match="/">
<html>
<body>
<h2>Students Database</h2>
<table border="1">
<tr bgcolor="gray">
<th>Name</th>
<th>Address</th>
<th>Standard</th>
<th>Marks</th>
</tr>
<xsl:for-each select="Student/Person-Details" >
<xsl:if test="marks>77">
<tr bgcolor="pink">
<td><xsl:value-of select="name"/></td>
<td><xsl:value-of select="address"/></td>
<td><xsl:value-of select="std"/></td>
<td><xsl:value-of select="marks"/></td>
</tr>
</xsl:if>
</xsl:for-each>
</table>
</body>
</html>
</xsl:template>
</xsl:stylesheet>
```

6. The <xsl:choose>

The <xsl:choose> element is used along with <xsl:when> and <xsl:otherwise> to express multiple conditional tests.

Step 1 : Keep the same xml[SimpleXml.xml] file which you have created in section 3.22.2 (1) Step 1.

Step 2 : Create the .xsl file as follows -

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
<xsl:template match="/">
<html>
<body>
<h2>Students Database</h2>
<table border="1">
<tr bgcolor="gray">
<th>Name</th>
<th>Marks</th>
</tr>
<xsl:for-each select="Student/Person-Details">
<tr>
<td><xsl:value-of select="name"/></td>
<xsl:choose>
<xsl:when test="marks < 77">
<td bgcolor="pink">
<xsl:value-of select="marks"/></td>
</xsl:when>
<xsl:otherwise>
<td><xsl:value-of select="marks"/></td>
</xsl:otherwise>
</xsl:choose>
</tr>
</xsl:for-each>
</table>
</body>
</html>
</xsl:template>
</xsl:stylesheet>
```

3.22.3 Difference between XML and XSLT

Sr.No.	XML	XSLT
1.	XML stands for extensible markup language.	XSLT stands for extensible stylesheet language transformations.
2.	XML is used to store data in structured format.	XSLT is used for transforming and formatting the XML file.
3.	XML does not perform transformations.	XSLT can transform one XML format into another XML format or into HTML or into plain text.
4.	XML contains the user defined tags.	XSLT contains special XSL tags and programming constructs.
5.	XML uses the user defined raw data for storage.	XSLT uses XPath for transformation and formatting of XML file.

Review Questions

1. Discuss the XSLT technology with an example
2. Write at least any five differences between XML and XSLT.

SPPU : May-19, Marks 5**SPPU : March-20, Marks 5****Part III : AJAX****3.23 Introduction**

- AJAX is a **Asynchronous Java Script** and XML
- Here
 - **Asynchronous** means, the execution of script does not disturb the user's work.
 - **JavaScript** because, it makes use of Javascripting to do the actual work.
 - **XML** because along with JavaScript the XML is also supported to perform the given task in AJAX.
- It is not a new programming language but it is a kind of web document which adopts certain standards.
- AJAX allows the developer to exchange the data with the server and updates the part of web document without reloading the web page.

3.24 Working of AJAX

- When user makes a request, the browser creates a object for the **HttpRequest** and a request is made to the server over an internet.

- The server processes this request and sends the required data to the browser.
- At the browser side the returned data is processed using JavaScript and the web document gets updated accordingly by sending the appropriate response.

Following Fig. 3.24.1 illustrates this working.

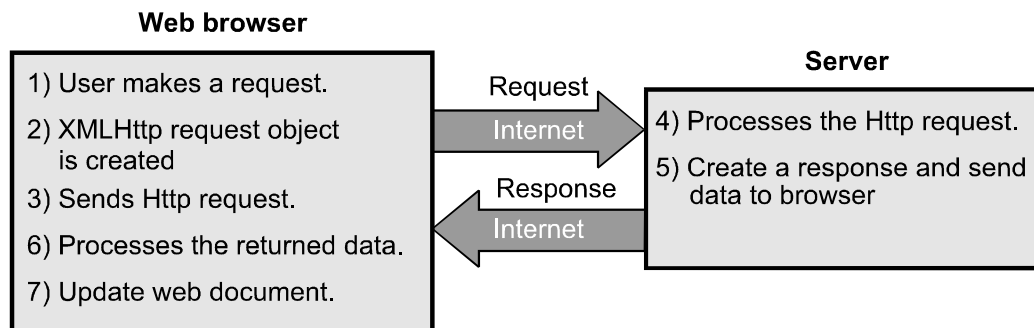


Fig. 3.24.1 Working of AJAX

Similarities and Differences

Sr.No.	Traditional Web Application Architecture	Ajax Based Web Application Architecture
1.	The web applications architectures are based on HTTP request response protocol.	
2.	At the client side the browser client has only one component and that is – user interface.	At the client side the browser client has user interface and AJAX Engine due to which the client gets quick response from the server.
3.	The architecture is based on client server communication.	
4.	It makes use of synchronous communication. The client has to wait to get the response from the server then only client can make the request to the server. It is start-stop-start-stop kind of communication.	By adding a new layer of AJAX Engine at the client side, it eliminates the start-stop-start-stop nature of communication and makes the communication asynchronous.
5.	With traditional web application architecture user cannot get rich user interface experience.	With AJAX architecture user gets rich user interface experience.
6.	It is less responsive.	It is more responsive.

7.	Building web application is simple.	The development time is more while designing the Ajax based web application.
8.	Limited use of JavaScript, CSS and XML technologies. In fact - Most user actions in the interface trigger an HTTP request back to a web server.	Extensive use of JavaScript, CSS and XML - i) standards-based presentation using XHTML and CSS; ii) Dynamic display and interaction using the Document Object Model(DOM); iii) Data interchange and manipulation using XML and XSLT; iv) Asynchronous data retrieval using XMLHttpRequest; v) JavaScript binding everything together

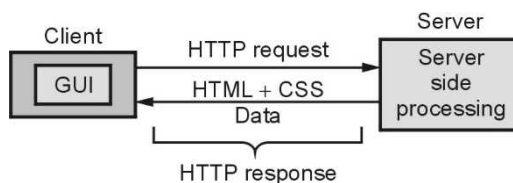


Fig. 3.24.2 Traditional web application architecture

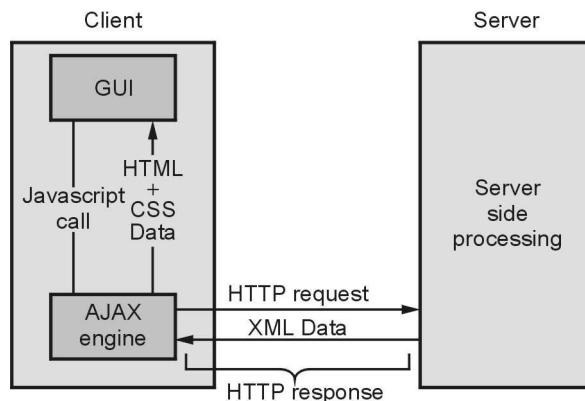


Fig. 3.24.3 AJAX web application architecture

Merits of AJAX

1. Response time is faster, hence increases performance and speed.
2. XMLHttpRequest object call as a asynchronous HTTP request to the Server for transferring data both side. It's used for making requests to the non-Ajax pages.
3. It is used in form validation.
4. It performs fetching of data from database and storing of data into database without reloading page.

Demerits of AJAX

1. It has browsing compatibility issues.
2. It is impossible to bookmark AJAX updated page contents.
3. Search engine would not crawl AJAX generated content. Hence, Search Engines like Google cannot index AJAX pages.

3.25 XMLHttpRequest Object

- XMLHttpRequest object is an important element in AJAX.
- XMLHttpRequest is an API which is used by JavaScript, VBScript and some other scripting languages.
- The methods of XMLHttpRequest are used in transferring data between a Web browser and a web server.
- It is because of XMLHttpRequest object, to update parts of web page without reloading the whole page.

Syntax

```
Variable = new XMLHttpRequest();
```

Various Methods of XMLHttpRequest object

Method	Purpose
new XMLHttpRequest()	Creates new XMLHttpRequest object.
Open(method, URL, async,user,pwd)	To make request this method is used. method: The request can be GET or POST URL: The location of file async: true(for asynchronous) and false(synchronous) user: user name(optional) pwd:password(optional)
send()	sends the request to the server.
send(string)	sends the request to the server.
setRequestHeader()	Adds the label- value pair to the header.

Various Properties of XMLHttpRequest object

Property	Description
readyState	Specifies the ready state of XMLHttpRequest 0: request not initialized 1: server connection established 2: request received 3: processing request 4: request finished and response is ready
onreadystatechange	When readyState property changes the onreadystatechange event listener will be automatically invoked
responseText	Returns the response data as a string
status	Returns the status number. Following is the meaning of various numbers returned by the status property 200: "OK" 403: "Forbidden" 404: "Not Found"
statusText	Returns the status text as "OK", or "Forbidden"

Example 3.25.1 *Create an XMLHttpRequest to retrieve data from an XML file and display the data in an HTML table. The data to be retrieved is a collection of stationary items stored in an XML file.*

Solution : Step 1 : Create an xml file storing the stationary items. The code is as follows -

```

/ELEMENTS>
<STATIONARY>
<ITEM>Notebook</ITEM>
<PRICE>40</PRICE>
</STATIONARY>
<STATIONARY>
<ITEM>Punching Machine</ITEM>
<PRICE>60</PRICE>
</STATIONARY>
<STATIONARY>
<ITEM>Pencil Box</ITEM>
<PRICE>40</PRICE>
</STATIONARY>

```

```
<STATIONARY>
<ITEM>Eraser</ITEM>
<PRICE>5</PRICE>
</STATIONARY>
</ELEMENTS>
```

Step 2 : The XML file displaying the stationary items in a tabular form is as given below -

```
test.html
<!DOCTYPE html>
<html>
<body>
<button type="button" onclick="loadXMLDoc()">Display Stationary Items</button>
<br><br>
<table id="demo" border="1"></table>
<script>
function loadXMLDoc()
{
  if(window.XMLHttpRequest)
  {
    xmlhttp = new XMLHttpRequest();
  }
  else
  {
    xmlhttp = new ActiveXObject("Microsoft.req");//to execute on Inter. Explorer
  }
  xmlhttp.onreadystatechange = function()
  {
    if (this.readyState == 4 && this.status == 200) {
      myFunction(this);
    }
  };
  xmlhttp.open("GET","stationary.xml",true);
  xmlhttp.send();
}
function myFunction(xml)
{
  var i;
  var xmlDoc = xml.responseXML;
  var table="<tr><th>Item</th><th>Price(Rs.)</th></tr>";
  var x = xmlDoc.getElementsByTagName("STATIONARY");
  for (i = 0; i <x.length; i++)
  {
    table += "<tr><td>" +
    x[i].getElementsByTagName("ITEM")[0].childNodes[0].nodeValue +
```

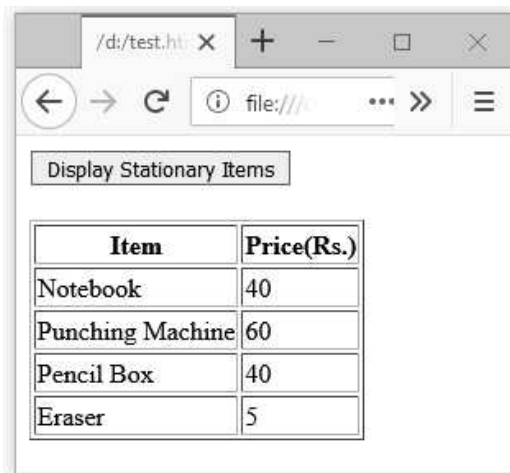


```

    "</td><td>" +
    x[i].getElementsByTagName("PRICE")[0].childNodes[0].nodeValue +
    "</td></tr>";
}
document.getElementById("demo").innerHTML = table;
}
</script>
</body>
</html>

```

Output



3.26 Coding AJAX Script

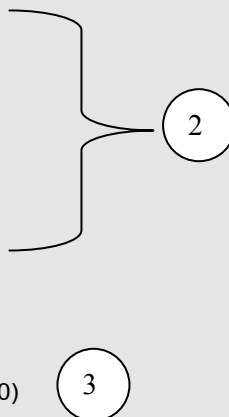
SPPU : Dec.-18, 19, May-18, 19, Marks 8

Let understand how AJAX works with the help of programming example

```

<html>
<head>
<script type="text/javascript">
function MyFun()
{
    if (window.XMLHttpRequest)
    {
        req=new XMLHttpRequest();
    }
    else
    {
        req=new ActiveXObject("Microsoft.req");
    }
    req.onreadystatechange=function()
    {
        if (req.readyState==4 && req.status==200)
        {

```



```

        document.getElementById("myID").innerHTML=req.responseText;
    }
}
req.open("GET","newdata.txt",true);
req.send();
}
</script>
</head>
<body>
<div id="myID">This text can be changed</div>

<button type="button" onclick="MyFun()">Change</button>
</body>
</html>

```

Script Explanation (Above numbered steps are explained here)

In above script, we have written some text which can be replaced by some another text on button click.

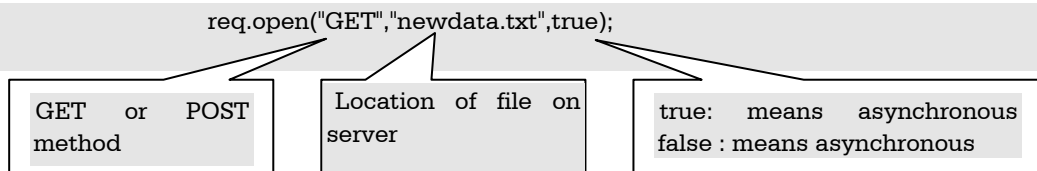
1. On button click a function **MyFun** is invoked. Thus client triggers the event.
2. **XMLHttpRequest** object is used to exchange data with a server. This object allows the user to change/update the parts of the web page without reloading it fully. The modern web browsers such as IE7+, FireFox, Chrome have built in **XMLHttpRequest** but old web browsers make use of **ActiveXObject**.
3. When a request to a server is sent, then **onreadystatechange** event is triggered.

The **readyState** property holds the status of the XMLHttpRequest. The **readyState=4** means request is finished and response is ready. The **status = 200** means "OK"

The DOM is modified and response is modified using the statement

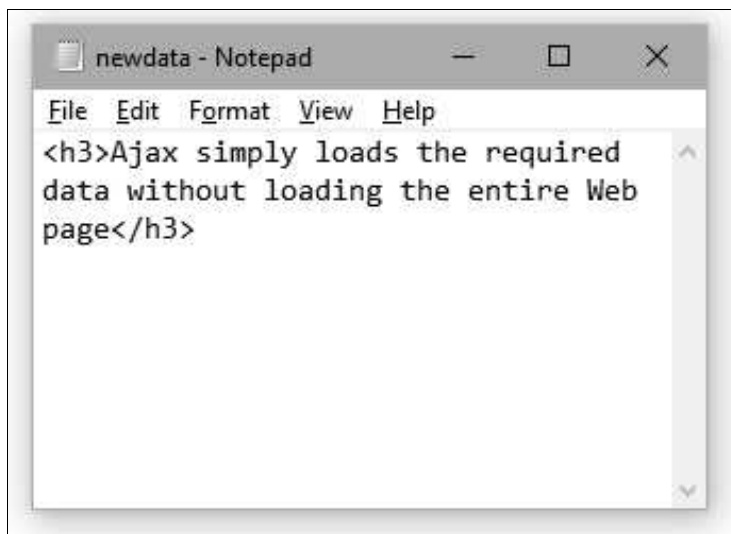
```
document.getElementById("myID").innerHTML=req.responseText;
```

4. The request can be sent to the server by using two functions **open()** and **send()**.

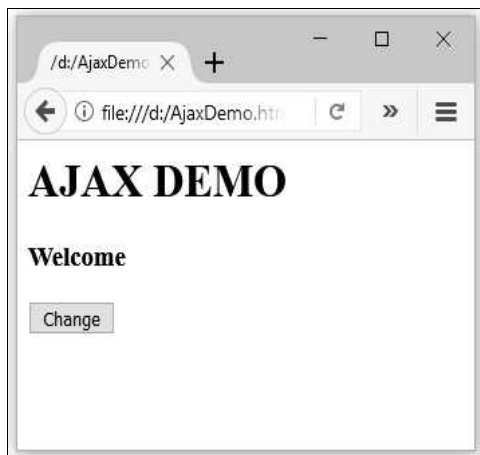


Asynchronous communication allows fast processing of the data.

The **newdata.txt** file contains some updating text using which our web page can be updated.

newdata.txt

5. The **send()** method sends the request to the server.

Output

Example 3.26.1 Write AJAX script to obtain the student information stored in XML document. The information should be displayed on clicking the button. It should be displayed in tabular form.

Solution :

Step 1 : Create an XML file for storing the student information. The XML file is as follows

Student.xml

```
<Student>
  <student_data>
    <Name>AAA</Name>
    <Marks>45</Marks>
  </student_data>
  <student_data>
    <Name>BBB</Name>
    <Marks>55</Marks>
  </student_data>
  <student_data>
    <Name>CCC</Name>
    <Marks>67</Marks>
  </student_data>
  <student_data>
    <Name>DDD</Name>
    <Marks>84</Marks>
  </student_data>
</Student>
```

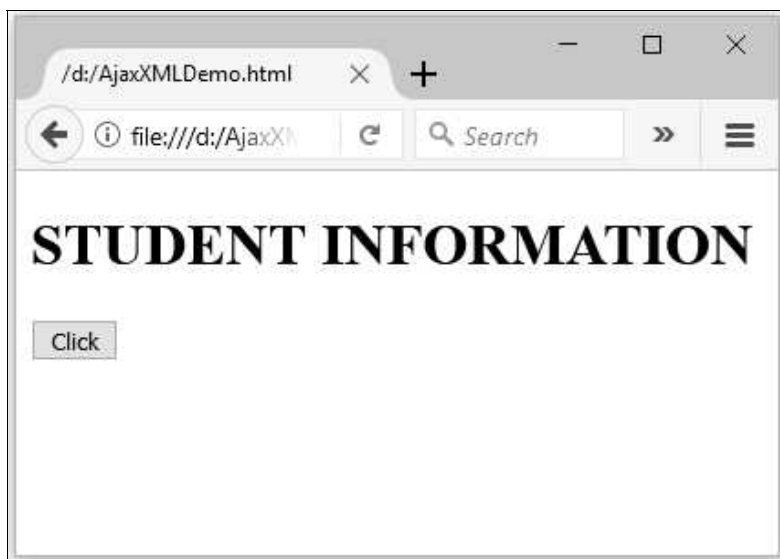
Step 2 : Create a AJAX script as follows -

AjaxXMLDemo.html

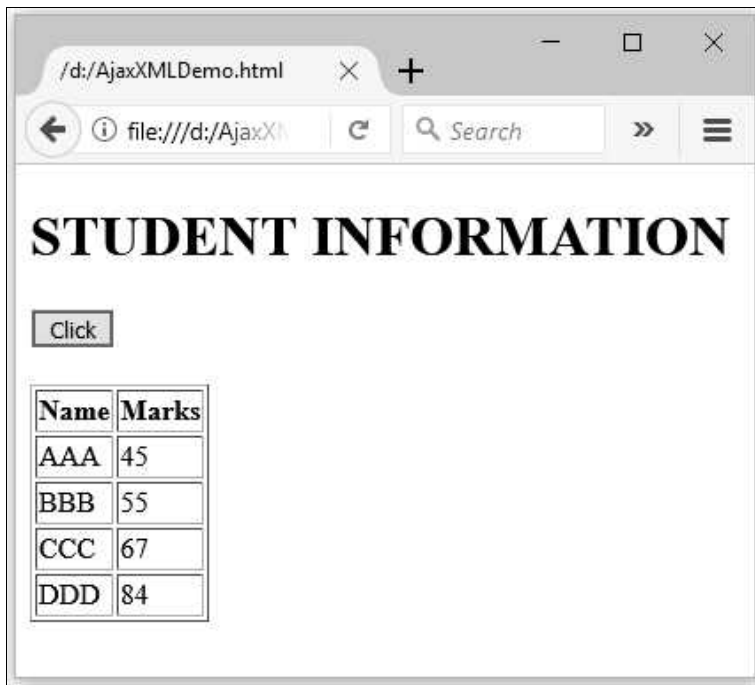
```
<!DOCTYPE html>
<html>
<body>
<h1>STUDENT INFORMATION</h1>
<button type="button" onclick="MyFun()">Click</button>
<br><br>
<table border="1" id="demo"></table>
<script>
function MyFun()
{
  var xhttp = new XMLHttpRequest();
  xhttp.onreadystatechange = function()
  {
    if (this.readyState == 4 && this.status == 200)
    {
```

```
        Load_XML_File(this);
    }
};
xhttp.open("GET", "Student.xml", true);
xhttp.send();
}
function Load_XML_File(xml)
{
    var i;
    var xmlDoc = xml.responseXML;
    var table="<tr><th>Name</th><th>Marks</th></tr>";
    var x = xmlDoc.getElementsByTagName("student_data");
    for (i = 0; i <x.length; i++)
    {
        table += "<tr><td>" +
            x[i].getElementsByTagName("Name")[0].childNodes[0].nodeValue +
            "</td><td>" +
            x[i].getElementsByTagName("Marks")[0].childNodes[0].nodeValue +
            "</td></tr>";
    }
    document.getElementById("demo").innerHTML = table;
}
</script>
</body>
</html>
```

Step 3 : Open the web browser and the output will be as follows -



On clicking the button we will get



Example 3.26.2 List and explain different values of ready state and status property of the *HttpRequestObject*.

SPPU : Dec.-18, Marks 6

Solution :

readyState	Holds the status of the XMLHttpRequest.
	0: request not initialized 1: server connection established 2: request received 3: processing request 4: request finished and response is ready
status	200: "OK" 400: "Bad Request" 403: "Forbidden" 404: "Page not found" 500: "Internal Server Error"

Review Questions

1. Draw and explain how AJAX works with the help of suitable example. **SPPU : May-18, Marks 6**
2. What is use of XMLHttpRequest object ? Explain its use with the help of simple JavaScript document. **SPPU : May-18, Marks 6**
3. Write different applications without AJAX and with AJAX. **SPPU : May-18, Marks 4**
4. Explain with simple example the XMLHttpRequest object. List and explain different values of readyState and status property of the XMLHttpRequest object. **SPPU : May-19, Marks 5**
5. What is the purpose of AJAX ? Explain with relevant example. **SPPU : Dec.-19, Marks 8**

3.27 Multiple Choice Questions

- Q.1 The Java ____ specification defines an application programming interface for communication between the web server and the application program.**
- | | |
|------------------------------------|--|
| ☐ a servlet | ☐ b server |
| ☐ c program | ☐ d none of these |
- Q.2 Servlet runs each request in ____.**
- | | |
|--|--|
| ☐ a JVM | ☐ b JRE |
| ☐ c servlet container | ☐ d none of these |
- Q.3 What is bytecode ?**
- | | |
|---|--|
| ☐ a Machine - specific code | ☐ b Java code |
| ☐ c Machine - independent code | ☐ d None of these |
- Q.4 What are the functions of Servlet container ?**
- | | |
|---|--|
| ☐ a Lifecycle management | ☐ b Communication support |
| ☐ c Multithreading support | ☐ d All of the above |
- Q.5 Which of the following package contains servlet classes ?**
- | | |
|--|---|
| ☐ a javax.servlet | ☐ b javax.servlet.http |
| ☐ c both a and b | ☐ d none of these |
- Q.6 Which method is used to specify before any lines that uses the PrintWriter ?**
- | | |
|--|---|
| ☐ a setPageType() | ☐ b setContentType() |
| ☐ c setResponseType() | ☐ d setContextType() |
- Q.7 What is the lifecycle of a servlet ?**
- | | |
|---|--|
| ☐ a Servlet class is loaded | ☐ b Servlet instance is created |
| ☐ c init, Service, destroy method is invoked | |
| ☐ d All of the above | |

- Q.8 What type of servlets use these methods doGet(), doPost(),doHead, doDelete(), doTrace() ?**
- | | |
|--|---|
| ☐ a) Generic Servlets | ☐ b) HttpServlets |
| ☐ c) All of the above | ☐ d) None of the above |
- Q.9 Which of the following code is used to get cookies in servlet ?**
- | | |
|---|--|
| ☐ a) response.getCookies() | ☐ b) request.getCookies() |
| ☐ c) Cookies.getCookies() | ☐ d) None of the above. |
- Q.10 If the client has disabled cookie in the browser which session management mechanism could the web container employ ?**
- | |
|---|
| ☐ a) Session management using URL rewriting |
| ☐ b) Session management using Cookies |
| ☐ c) Either Cookies or URL rewriting |
| ☐ d) Cookies and URL rewriting must be used together |
- Q.11 Servlet is initialized by calling the following method :**
- | | |
|---------------------------------------|---------------------------------------|
| ☐ a) service() | ☐ b) destroy() |
| ☐ c) init() | ☐ d) none |
- Q.12 Common gateway interface is used to :**
- | |
|--|
| ☐ a) generate executable files from web content by web server |
| ☐ b) generate web pages |
| ☐ c) stream video |
| ☐ d) none |
- Q.13 The Java _____ specification defines an application programming interface for communication between the web server and the application program.**
- | | |
|-------------------------------------|---------------------------------------|
| ☐ a) Servlet | ☐ b) Server |
| ☐ c) Program | ☐ d) Randomize |
- Q.14 Which JDBC driver type(s) can be used in either applet or servlet code.**
- | | |
|--|--|
| ☐ a) Both type 1 and type 2 | ☐ b) Both type 1 and type 3 |
| ☐ c) Both type 3 and type 4 | ☐ d) Type 4 only |
- Q.15 XML stands for ____ .**
- | | |
|--|---|
| ☐ a) Extensible Markup Language | ☐ b) Extended Markup Language |
| ☐ c) XYZ Markup Language | ☐ d) Exclusive Markup Language |

Q.16 Correct version of the declaration which defines the XML version ____ .

☐ a <?xml version="1.0"/>

☐ b ?xml version="1.0"?>

☐ c <xml version="1.0">

☐ d none of these

Q.17 XML is ____ .

☐ a platform independent

☐ b language independent

☐ c both a and b

☐ d none of these

Q.18 Which language is case sensitive ?

☐ a HTML

☐ b XML

☐ c Both a and b

☐ d None of these

Q.19 DTD stands for ____ .

☐ a Document Trial Data

☐ b Document Trial Definition

☐ c Data Type Definition

☐ d Document Type Definition

Q.20 XML DOM is a standard for performing the following operation on XML elements ____.

☐ a change

☐ b add

☐ c delete

☐ d all the above

Q.21 DOM parser is used on ____ .

☐ a client side

☐ b server side

☐ c both a and b

☐ d none

Q.22 Which are techniques for defining the structure of a specific type of XML documents ?

☐ a Schema

☐ b DTD

☐ c Both a and b

☐ d none of the above

Q.23 Which statement is true ?

☐ a All the statements are true

☐ b All XML elements must have a closing tag

☐ c All XML elements must be lower case

☐ d All XML documents must have a DTD

Q.24 DTD includes the specifications about the markup that can be used within the document, the specifications consists of all EXCEPT ____.

☐ a the browser name

☐ b the size of element name

☐ c entity declarations

☐ d element declarations

Q.25 Parameter entities can appear in ____.

- ☐ a xml file ☐ b dtd file
☐ c xsl file ☐ d both 1 and 2

Q.26 Which characters are illegal in XML elements ?

- ☐ a > ☐ b &
☐ c Both a and b ☐ d None of these

Q.27 The syntax for parameter entity is ____ .

- ☐ a <! ENTITY % NAME DEFINITION>
☐ b < ENTITY % NAME DEFINITION>
☐ c <! ENTITY \$ NAME DEFINITION>
☐ d < ENTITY \$ NAME DEFINITION>

Q.28 The XML DOM object is ____ .

- ☐ a entity ☐ b entity reference
☐ c comment reference ☐ d comment data

Q.29 Which of the following is not a XML storage option ?

- ☐ a Native storage as XML data type
☐ b Mapping between XML and relational storage
☐ c Small object storage
☐ d None of the mentioned

Q.30 Which of the following part of the XML data stored in an XML column is very important for locking ?

- ☐ a Granularity ☐ b Degree of structure
☐ c Hierarchy ☐ d None

Answer Keys for Multiple Choice Questions :

Q.1	a	Q.2	c	Q.3	c	Q.4	d	Q.5	c
Q.6	b	Q.7	d	Q.8	b	Q.9	b	Q.10	a
Q.11	c	Q.12	a	Q.13	a	Q.14	c	Q.15	a
Q.16	b	Q.17	c	Q.18	b	Q.19	c	Q.20	d
Q.21	c	Q.22	c	Q.23	b	Q.24	a	Q.25	b
Q.26	c	Q.27	a	Q.28	b	Q.29	c	Q.30	a

